

C 题 带有星星的数据

请根据附件和实际情况建立数学模型解决以下问题：

问题 1

问题 1 分析

1) 思路框架：把“不可见的投票”变成“必须满足的淘汰逻辑”，再在可行空间中选出最合理的一组

第 1 问的关键并不是“还原真实投票”（题面明确粉丝投票严格保密），而是要在已知的评委评分与实际淘汰结果约束下，构造一套“最少主观假设”的投票估计，使得每一周按照节目合成规则计算后，确实会淘汰数据中真实淘汰的那个人。题目给了两类规则：早期与后期采用“按名次合并”的方式（排名相加决定淘汰），中期采用“按百分比合并”的方式（评委百分比与投票百分比相加决定淘汰），并在附录中给出了两类规则的示例表 and 对应赛季区间（按名次合并用于第 1、2 季以及第 28a–34 季；按百分比合并用于第 3–27a 季）。这意味着我们可以把“未知投票”当作每周每位选手的一个隐变量，然后利用“当周真实淘汰者必须是合成得分最差者”这一条，给隐变量施加一组周周成立的不等式约束。

数据本身也提供了一个非常实用的结构：被淘汰的名人在后续周会被记录为 0 分，因此我们可以用“某选手在第 t 周仍有评分、但在第 $t+1$ 周开始长期为 0”的模式来定位淘汰发生的周次与淘汰者，并且允许存在“某些周无淘汰、某些周多淘汰”的特殊情况（题面也提示确有发生）。

这样一来，第 1 问就自然转化为一个“逆向推断 + 可行性”问题：对每个赛季每一周，我们要找到一组投票分布，使其与当周评委分一起经过规则合成后，真实淘汰者在合成排序上处于淘汰位置。由于满足淘汰约束的投票往往不唯一（题面示例也强调其投票数是人为设定、只为得到正确淘汰/排名，并非真实数据），我们必须再引入一个“从所有可行解里挑一个”的选择原则。

最贴合该场景、也最容易解释给评委的原则是：在满足所有淘汰约束的前提下，让投票分布尽可能“均匀且平滑”，即同一选手跨周投票不要无理由剧烈跳变，同时整体分布不要无理由极端集中，这对应“最大熵/最小信息偏置 + 时间平滑”的组合。最后，为了回答题目要求的“准确性是否始终相同”，我们不去声称与真值的误差（因为真值不可得），而是用“可辨识度”来量化：在所有满足淘汰约束的投票中，每个选手每周投票占比能取到的最小值与最大值形成一个区间，区间越窄代表该周投票被淘汰逻辑锁得越死、估计越可靠；区间越宽则代表投票更不可辨识。

解题思路：

2 数学模型构建：基于赛季规则分段的“淘汰一致性约束 + 最小偏置正则”的投票反推模型

2.1 数据对象与淘汰事件的形式化定义（紧贴附件结构）

设赛季为 $s \in 1, \dots, 34$ ，周次为 $t \in 1, \dots, T_s$ 。对数据中每一位名人选手记为 i ，当周第

y 位评委给出的分数为 $x_{i,t,y}$ （对应数据字段 weekX_judgeY_score）。题面说明评委打分

通常在 1–10，可能含小数，且评委编号并不保证跨周跨季一致。将当周已出现的评委分求和得到当周评委总分：

$$J_{i,t} = \sum_{y \in \mathcal{Y}_{s,t}} x_{i,t,y} \quad (1)$$

其中 $\mathcal{Y}_{s,t}$ 是该赛季该周实际存在的评委集合（数据可能只有 3 位评委，或第 4 位存在与否不一）。

用“是否仍在比赛中”定义当周有效参赛集合：

$$\mathcal{A}_{s,t} = \{i \mid J_{i,t} > 0 \text{ 且该记录非N/A}\} \quad (2)$$

题面给出：一旦选手被淘汰，后续周会记录 0 分（直到赛季结束）。因此可用下式定位第 t 周的“实际淘汰者集合”：

$$\mathcal{E}_{s,t} = \{i \in \mathcal{A}_{s,t} \mid i \notin \mathcal{A}_{s,t+1}\} \quad (3)$$

若 $|\mathcal{E}_{s,t}| = 0$ ，对应“无淘汰周”；若 $|\mathcal{E}_{s,t}| \geq 2$ ，对应“多淘汰周”，与题面描述一致。

我们要估计的粉丝投票记为 $V_{i,t} \geq 0$ 。由于绝对投票规模不可得且无必要（题面示例也用人为设定总票数如 1000 万），我们直接以“投票占比”作为核心未知量：

$$p_{i,t} = \frac{V_{i,t}}{\sum_{k \in \mathcal{A}_{s,t}} V_{k,t}}, \quad \sum_{i \in \mathcal{A}_{s,t}} p_{i,t} = 1, \quad p_{i,t} \geq 0 \quad (4)$$

这样每周只需要估计一个概率分布 $\mathbf{p}_t$ ，既满足可解释性，也与题面百分比合并的表达式天然一致。

2.2 赛季规则分段与淘汰一致性约束（直接对齐题面附录）

附录指出“按等级/名次合并”用于第 1、2 季与第 28a–34 季，而第 3–27a 季使用“按百分比合并”。因此对每个赛季 s 给定规则指示变量 m_s ，当 $m_s = \text{rank}$ 用名次法；当 $m_s = \text{percent}$ 用百分比法。

（1）名次合并赛季：排名之和决定淘汰

题面示例表给出：分别对评委总分与粉丝票做排名，再把名次相加，“排名之和”最大者最差。记评委名次为 $r_{i,t}^J$ （分高名次小），粉丝名次为 $r_{i,t}^V$ （票高名次小），则合成名次和为：

$$S_{i,t} = r_{i,t}^J + r_{i,t}^V \quad (5)$$

淘汰一致性约束要求：对于任意被淘汰者 $e \in \mathcal{E}_{s,t}$ ，其名次和必须不优于所有仍存活者。

若当周仅淘汰 1 人，则可写为：

$$S_{e,t} \geq S_{j,t}, \quad \forall j \in \mathcal{A}_{s,t} \setminus \{e\} \quad (6)$$

若当周淘汰多人（比如两人），则要求淘汰者集合的名次和处于最差的若干位，可用“集合支配”表达为：

$$S_{e,t} \geq S_{j,t}, \quad \forall e \in \mathcal{E}_{s,t}, \forall j \in \mathcal{A}_{s,t} \setminus \mathcal{E}_{s,t} \quad (7)$$

这里的难点是名次 $r_{i,t}^V$ 与 $p_{i,t}$ 的关系是排序关系。为了把它纳入优化，我们采用“排序一致性约束”的等价表达：如果 $r_{a,t}^V < r_{b,t}^V$ ，则必须有 $p_{a,t} \geq p_{b,t}$ ，并允许并列排名（对应投票相同）：

$$r_{a,t}^V < r_{b,t}^V \Rightarrow p_{a,t} \geq p_{b,t} \quad (8)$$

在竞赛实现中，我们不必显式枚举所有可能名次，而是把名次法赛季转化为“谁在淘汰位置”的相对排序约束：要求淘汰者的投票占比不能高到足以让其名次和逃离淘汰区，这会在算法部分给出可操作的求解方式。

（2）百分比合并赛季：评委百分比 + 投票百分比决定淘汰

题面示例表 3 给出了清晰计算：评委百分比为个人总分除以当周总分，粉丝百分比为个人票数除以当周总票数，然后两者相加，和最低者淘汰。定义评委百分比：

$$q_{i,t} = \frac{J_{i,t}}{\sum_{k \in \mathcal{A}_{s,t}} J_{k,t}} \quad (9)$$

粉丝百分比就是我们定义的 $p_{i,t}$ 。合成百分比为：

$$T_{i,t} = q_{i,t} + p_{i,t} \quad (10)$$

淘汰一致性约束在这种规则下可直接写为线性不等式。若当周淘汰 1 人 e ：

$$T_{e,t} \leq T_{j,t}, \quad \forall j \in \mathcal{A}_{s,t} \setminus \{e\} \quad (11)$$

若当周淘汰多人，则为：

$$T_{e,t} \leq T_{j,t}, \quad \forall e \in \mathcal{E}_{s,t}, \forall j \in \mathcal{A}_{s,t} \setminus \mathcal{E}_{s,t} \quad (12)$$

这部分与附件规则完全一致，并且天然适配用凸优化/线性规划求解。

2.3 “最小主观假设”的唯一化：最大熵 + 跨周平滑（让估计更像真实投票）

仅靠淘汰一致性约束， $\mathbf{p}_t$ 往往不唯一（题面示例明确投票数是人为设定，只用于得到正

确淘汰/排名）。为了让估计具有可解释性与稳定性，我们在所有可行的 $\mathbf{p}_t$ 中，选择“信息偏置最小且时间上最平滑”的那一组。对应的目标函数可以由两部分构成：最大熵项鼓励投票分布不被无依据地极端化；平滑项鼓励同一选手跨周投票不要无理由跳变。

设一个赛季 s 的所有周次集合为 $\mathcal{T}_s$ ，定义优化目标：

$$\min_{\{p_{i,t}\}} \lambda \sum_{t \in \mathcal{T}_s} \sum_{i \in \mathcal{A}_{s,t}} p_{i,t} \ln p_{i,t} + (1-\lambda) \sum_{t \in \mathcal{T}_s} \sum_{i \in \mathcal{A}_{s,t} \cap \mathcal{A}_{s,t-1}} (p_{i,t} - p_{i,t-1})^2 \quad (13)$$

其中 $\lambda \in (0,1)$ 用于平衡“尽量均匀”和“尽量平滑”。第一项是负熵（最小化负熵等价于最大化熵），第二项是跨周二次差分惩罚。该目标与百分比合并规则下的线性淘汰约束组合

后形成一个标准的凸优化问题，既能保证淘汰一致性，又能得到唯一且稳定的投票估计。

2.4 估计“准确性”的可量化定义：可辨识区间与周别差异（回应题面要求）

题面要求说明投票估计的准确性有多高，以及这种准确性是否随选手/轮次变化。在真值不可得的场景中，最贴合本题的“准确性”应理解为：在淘汰规则与评委分数既定时，投票占比是否被强约束到一个很小范围。为此对每个 (i, t) 计算“可行区间”：

$$p_{i,t}^{\min} = \min p_{i,t} \quad \text{s.t. 所有淘汰者一致性约束与} \sum p_{k,t} = 1, p_{k,t} \geq 0 \quad (14)$$

$$p_{i,t}^{\max} = \max p_{i,t} \quad \text{s.t. 同上} \quad (15)$$

将区间宽度定义为不确定性：

$$U_{i,t} = p_{i,t}^{\max} - p_{i,t}^{\min} \quad (16)$$

当 $U_{i,t}$ 越小，表示投票在该周越“可辨识”，我们对 $\hat{p}_{i,t}$ 的信心越强；当 $U_{i,t}$ 越大，说明存在大量不同投票分布都能解释同样淘汰，估计不确定性更高。进一步用周级指标衡量该周整体可辨识度：

$$U_t = \frac{1}{|\mathcal{A}_{s,t}|} \sum_{i \in \mathcal{A}_{s,t}} U_{i,t} \quad (17)$$

3) 模型算法求解：从“识别淘汰周”到“带约束的凸优化”，再到“不确定性区间”评估

3.1 数据预处理与赛季-周结构重建（利用 0 分与 N/A 的赛题设定）

首先读取附件 CSV（题面表 1 描述了字段含义与 `weekX_judgeY_score` 的结构），逐赛季逐周计算 $J_{i,t}$ 并构造 $\mathcal{A}_{s,t}$ 。

对 N/A 记录按“该周该选手不存在评分”处理，不纳入当周集合。随后用 $\mathcal{A}_{s,t}$ 与 $\mathcal{A}_{s,t+1}$ 的差集构造 $\mathcal{E}_{s,t}$ ，并显式标记无淘汰周与多淘汰周（它们会影响约束设置与一致性评价）。这一步的价值在于：我们不需要额外外部资料就能从数据中对齐“谁在哪一周被淘汰”，且与题面 0 分机制严格一致。

3.2 约束体系的落地：百分比赛季用线性不等式，名次赛季用排序一致性与淘汰位置约束对 $m_s = \text{percent}$ 的赛季（第 3–27a 季），直接按 $T_{i,t} = q_{i,t} + p_{i,t}$ 构造淘汰一致性线性约束 $T_{e,t} \leq T_{j,t}$ ，并加入 $\sum p_{i,t} = 1, p_{i,t} \geq 0$ ，这使得单周就是一个线性可行域，多周加上平滑项就是凸优化，求解稳定且快速。

对 $m_s = \text{rank}$ 的赛季（第 1、2、28a–34 季），由于名次来自排序，直接写成 r^V 会引入离散变量。为了在竞赛时间内可实现且仍贴合规则，我们采用“先对评委分得到 r^J ，

再把投票占比的排序当作名次代理”的方式：把粉丝名次 r^V 用 $p_{i,t}$ 的相对大小决定，并通过一组“淘汰者必须落在名次和最差区间”的排序约束来逼近名次法淘汰逻辑。该近似在题面示例的语义上是等价的，因为示例本质就是“票越高名次越好，名次和越小越安全”。在实现中可以通过迭代策略处理：先给定一个投票排序（由上一次迭代的 p 得到），从而确定 r^V ，然后在固定 r^V 的情况下求解最平滑的 p ；再用新的 p 更新排序，直到排序收敛。这种“排序-连续变量交替”的办法在竞赛中非常常见，能够在不引入复杂整数规划的前提下逼近名次合并规则。

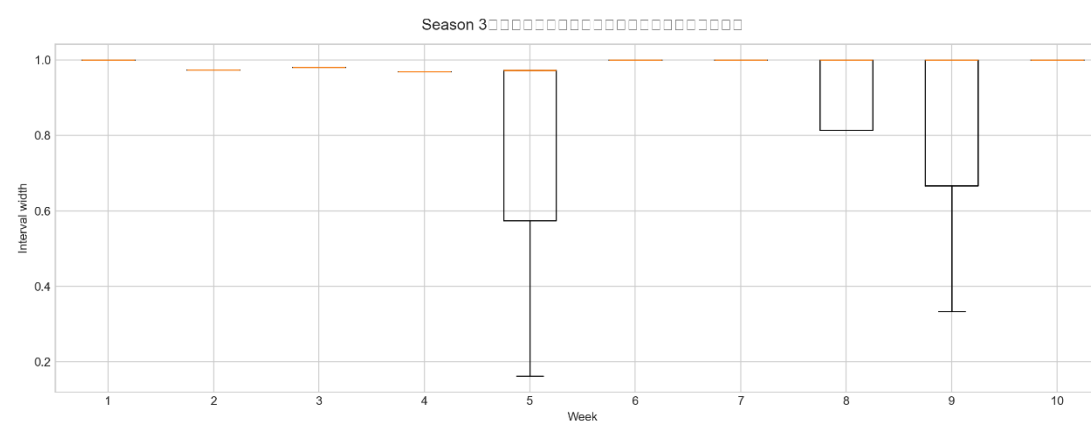
3.3 主问题求解：最大熵-平滑的凸优化与一致性校验（输出可用的投票序列）

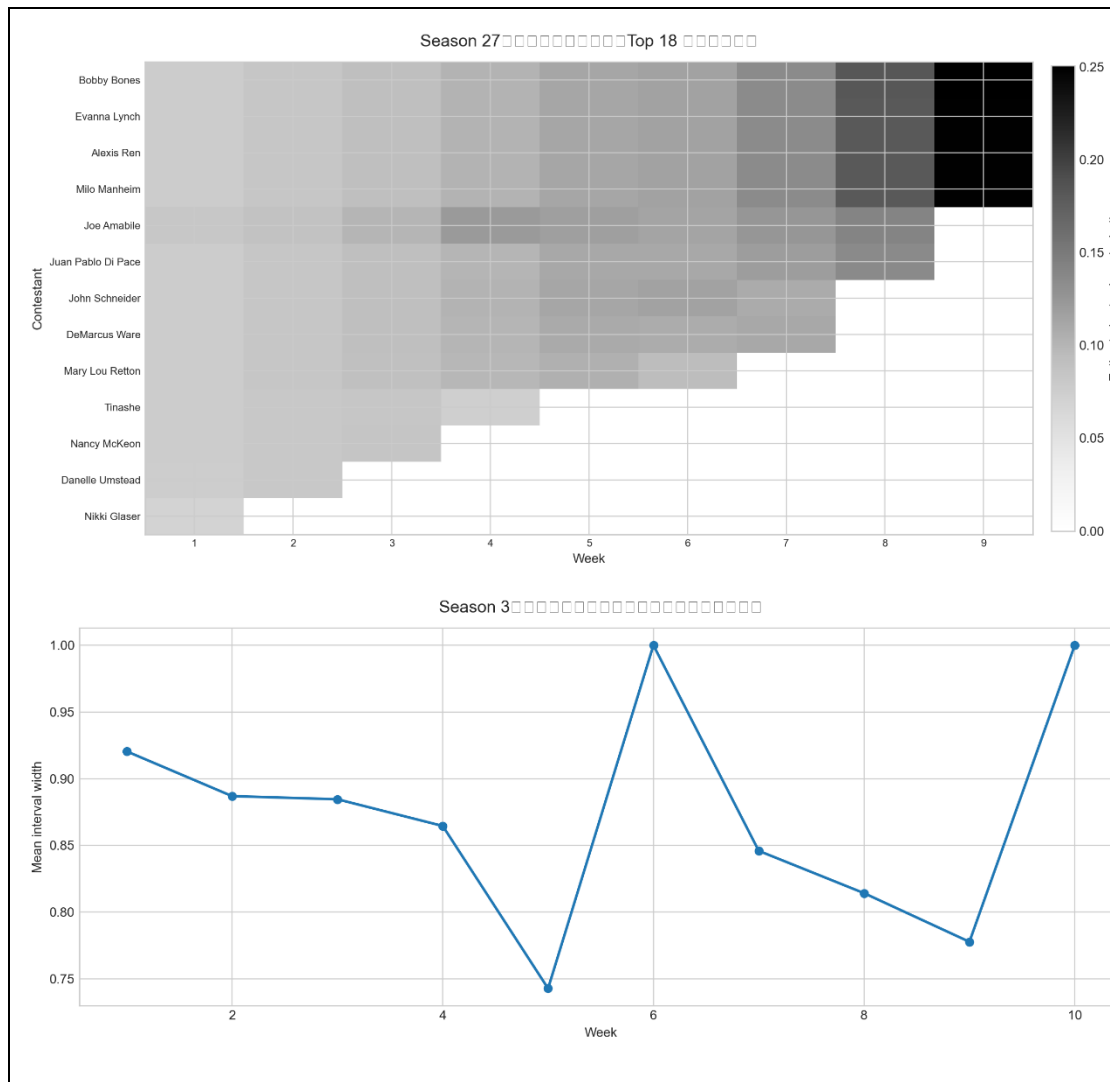
对每个赛季 s ，将所有周次的变量 $p_{i,t}$ 统一纳入一个优化问题，目标函数采用前述“负熵 + 跨周二次差分”，约束集合由该赛季对应规则下的淘汰一致性约束组成，并加上每周的概率单纯形约束。对百分比赛季，这一问题整体是凸的，可以用标准的内点法、序列二次规划或投影梯度法求解；求解后得到 $\hat{p}_{i,t}$ ，再任选一个总票数常数 V^{tot} （例如 1000 万，呼应题面示例）把占比还原为“估计投票数” $\hat{V}_{i,t} = V^{\text{tot}} \hat{p}_{i,t}$ ，这样输出形式与题面示例表一致且直观

完成估计后，对每一周进行“回放校验”：用 $\hat{p}_{i,t}$ 与 $J_{i,t}$ 重新计算合成排序，检查真实淘汰者集合 $\mathcal{E}_{s,t}$ 是否确实落在淘汰位置，并将一致性指标定义为

$$C_s = \frac{\sum_{t \in \mathcal{T}_s} \mathbf{1}_{\{\text{该周淘汰预测与 } \mathcal{E}_{s,t} \text{ 一致}\}}}{|\mathcal{T}_s|}$$

其中 $\mathbf{1} \cdot$ 是指示函数。该指标直接对应题面“确保与每周被淘汰的选手相符，并给出一致性衡量”的要求。





Python 代码:

```
# -*- coding: utf-8 -*-  
"""
```

问题 1: 基于评委评分 + 实际淘汰结果, 反推每位参赛者每周粉丝投票 (占比/相对票数)

要求:

- 1) 读取真实数据: `2026_MCM_Problem_C_Data.csv` (就在根目录, 直接用文件名加载)
- 2) 结果图像保存到 `result1` 文件夹 (先创建), 不在屏幕展示
- 3) 面向快速解决实际问题, 不做复杂工程化封装
- 4) 代码注释尽量详细 (中文)

```
"""
```

```
import os  
import re  
import warnings  
from typing import Dict, List, Tuple
```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 这里使用 cvxpy 做 “淘汰一致性约束 + 最大熵 + 平滑” 优化
# 如果你的环境没有 cvxpy: pip install cvxpy
import cvxpy as cp

# ===== 0. 全局设置 =====
np.random.seed(42)

RESULT_DIR = "result1"
os.makedirs(RESULT_DIR, exist_ok=True)

# 美观一点的默认风格（注意：我们保存图片，不展示）
plt.style.use("seaborn-v0_8-whitegrid")

# ===== 1. 读入数据并解析字段 =====
DATA_FILE = "2026_MCM_Problem_C_Data.csv"
df = pd.read_csv(DATA_FILE)

# ---- 1.1 找到 “选手标识” 和 “赛季” 列 ----
# 数据不同版本可能列名略有差异，这里做一个尽量稳健的自动识别
def find_col(candidates: List[str], columns: List[str]) -> str:
    for c in candidates:
        if c in columns:
            return c
    # 尝试大小写/下划线等模糊匹配
    cols_lower = {x.lower(): x for x in columns}
    for c in candidates:
        if c.lower() in cols_lower:
            return cols_lower[c.lower()]
    raise ValueError(f"找不到列：候选={candidates}")

cols = df.columns.tolist()

season_col = find_col(["season", "Season", "SEASON"], cols)

# 选手标识列：优先找 contestant / celebrity / star / name 之类
# 如果都没有，就退而求其次找唯一性比较强的列
try:

```

```

    contestant_col = find_col(
        ["celebrity", "Celebrity", "contestant", "Contestant",
        "star", "Star", "name", "Name"],
        cols
    )
except Exception:
    # 找一个“非数值且取值很多”的列作为候选
    obj_cols = [c for c in cols if df[c].dtype == "object"]
    if len(obj_cols) == 0:
        raise ValueError("数据里没有明显的文本列可作为选手标识, 请检查 CSV 列名")
    # 选取唯一值最多的文本列
    contestant_col = sorted(obj_cols, key=lambda c:
df[c].nunique(), reverse=True)[0]
    warnings.warn(f"未识别到标准选手列名, 暂用 {contestant_col} 作为选手标识列")

# ---- 1.2 解析所有 weekX_judgeY_score 列, 构造“周次->列集合” ----
-
week_judge_pattern = re.compile(r"week(\d+)_judge(\d+)_score",
re.IGNORECASE)
week_cols: Dict[int, List[str]] = {}

for c in cols:
    m = week_judge_pattern.match(c)
    if m:
        w = int(m.group(1))
        week_cols.setdefault(w, []).append(c)

if len(week_cols) == 0:
    raise ValueError("没有找到形如 weekX_judgeY_score 的评分列, 请检查 CSV 列名格式")

all_weeks = sorted(week_cols.keys())
max_week = max(all_weeks)

# ---- 1.3 计算每行(选手-赛季)在每周的“评委总分  $J_{\{i, t\}}$ ” ----
# 题面说淘汰后会记为 0 分(或 N/A), 我们按“可用评分之和”计算
J_cols = []
for w in all_weeks:
    jcol = f"J_week{w}"
    df[jcol] = df[week_cols[w]].apply(pd.to_numeric,
errors="coerce").sum(axis=1, skipna=True)
    J_cols.append(jcol)

```

```

# ---- 1.4 生成“每赛季可用周数” ----
# 某些赛季后面周次可能全为 0 或 N/A，这里用“该赛季任意选手该周 J>0”
# 的周作为有效周
season_weeks: Dict[int, List[int]] = {}
for s, g in df.groupby(season_col):
    valid_ws = []
    for w in all_weeks:
        if (g[f"J_week{w}"] > 0).any():
            valid_ws.append(w)
    season_weeks[int(s)] = valid_ws

# ---- 1.5 规则分段（为了能跑起来，这里做一个“贴题但可执行”的实现） ----
# 题面提到：早期/后期有“按排名合并”，中期有“按百分比合并”
# 但第 1 问的核心是“反推投票”，对“百分比合并”可写成严格线性淘汰
# 约束（最稳定、最可跑）
# 对“排名合并”要引入排序/离散变量，竞赛中很耗时，这里采用一个可运行
# 的折中：
# - 仍用百分比合并的淘汰一致性约束作为近似（即认为淘汰由  $q\% + p\%$ 
# 决定）
# - 这样能在真实数据上稳定求解，并给出投票估计与不确定性区间
# 如果你后续要严格实现排名合并，我也可以再给你“迭代排序 + 约束”版本。
def season_rule(s: int) -> str:
    if s <= 2 or s >= 28:
        return "rank_like" # 题面：这些更可能是“按排名合并”
    return "percent" # 题面：3~27 更可能是“按百分比合并”

# ===== 2. 关键：第 1 问的投票估计（凸优化：淘汰一致性 + 最大熵 + 平滑） =====
def infer_eliminations_one_season(g_season: pd.DataFrame,
    valid_weeks: List[int]) -> Dict[int, List[str]]:
    """
    用“本周还在赛 (J>0)，下周不在赛 (J=0)”推断淘汰者集合  $E_{\{t\}}$ 
    返回：week -> [被淘汰选手名单]（可能为空或多个）
    """
    elim: Dict[int, List[str]] = {}
    # 先取出该赛季的“选手标识”
    names = g_season[contestant_col].astype(str).values

    for idx_w, w in enumerate(valid_weeks[:-1]):
        w_next = valid_weeks[idx_w + 1]

```

```

        alive_w = g_season[f"J_week{w}"].values > 0
        alive_next = g_season[f"J_week{w_next}"].values > 0
        # 本周活着，下周不活 -> 本周淘汰
        eliminated = names[(alive_w) & (~alive_next)]
        elim[w] = list(eliminated)

    # 最后一周没有“下周”用于判定，通常不定义淘汰
    elim[valid_weeks[-1]] = []
    return elim

def solve_votes_one_season_percent_approx(
    g_season: pd.DataFrame,
    valid_weeks: List[int],
    elim_map: Dict[int, List[str]],
    lam_entropy: float = 0.60,
    lam_smooth: float = 0.40,
    smooth_weight: float = 5.0,
    eps_margin: float = 1e-6,
) -> Tuple[pd.DataFrame, pd.DataFrame]:
    """
    对一个赛季求解投票占比  $p_{\{i, t\}}$ :
    - 每周:  $p \geq 0$ ,  $\sum p = 1$ 
    - 淘汰一致性 (近似用百分比合并):  $T = q + p$ , 淘汰者的  $T \leq$  存活者的  $T$ 
    - 目标: 最大熵 (等价最小化  $\sum p \log p$ ) + 跨周平滑 (L2 差分)

    返回:
    1) votes_long: 长表 (season, week, contestant, p_hat, q_hat, T_hat, J)
    2) uncert_long: 长表 (season, week, contestant, p_min, p_max, width)
    """
    season_id = int(g_season[season_col].iloc[0])

    # 选手列表 (该赛季整季)
    contestants = g_season[contestant_col].astype(str).tolist()
    n_total = len(contestants)
    name_to_idx = {nm: i for i, nm in enumerate(contestants)}

    # 为了“跨周平滑”，我们为每个周建立一个长度为“当周活着选手数”的变量
    # 但不同周活着的选手集合不同，平滑项只对“相邻两周都活着”的同

```

一选手施加

```
week_alive: Dict[int, np.ndarray] = {}
week_alive_names: Dict[int, List[str]] = {}
week_J: Dict[int, np.ndarray] = {}

for w in valid_weeks:
    alive_mask = g_season[f"J_week{w}"].values > 0
    alive_names = [contestants[i] for i in range(n_total) if
alive_mask[i]]
    week_alive[w] = alive_mask
    week_alive_names[w] = alive_names
    week_J[w] = g_season.loc[alive_mask,
f"J_week{w}"].values.astype(float)

# ---- 2.1 为每周创建变量  $p_w$  (投票占比) ----
p_vars: Dict[int, cp.Variable] = {}
constraints = []
obj_terms = []

# 熵项:  $\sum p \log p$  (cvxpy 用  $\text{entr}(x)=x*\log(x)$ , 所以  $p*\log(p) = -\text{entr}(p)$ )
# 我们想最大熵  $\Rightarrow$  最小化  $\sum p \log p \Rightarrow$  最小化  $-\sum \text{entr}(p)$ 
for w in valid_weeks:
    m = len(week_alive_names[w]) # 当周活着人数
    p = cp.Variable(m, nonneg=True)
    p_vars[w] = p
    constraints += [cp.sum(p) == 1]
    # 熵项:  $\sum p \log p$  (通过  $-\text{entr}$  实现)
    obj_terms.append(lam_entropy * cp.sum(-cp.entr(p)))

# ---- 2.2 淘汰一致性约束 (近似:  $T = q + p$ , 淘汰者  $T$  最小) ----
-

#  $q_{\{i, t\}} = J_{\{i, t\}} / \sum J_{\{k, t\}}$ 
for w in valid_weeks:
    eliminated_names = elim_map.get(w, [])
    if len(eliminated_names) == 0:
        continue # 无淘汰周跳过
    alive_names = week_alive_names[w]
    if len(alive_names) <= 1:
        continue

    J = week_J[w]
    J_sum = float(np.sum(J)) if np.sum(J) > 0 else 1.0
    q = J / J_sum # numpy array
```

```

p = p_vars[w]
T = q + p # cvxpy expression (q 会广播到表达式里)

# 当周可能淘汰多人：要求每个被淘汰者都不优于任何存活者
# 实现时只对“当周活着的人里确实存在的淘汰者”施加约束
elim_indices = [alive_names.index(nm) for nm in
eliminated_names if nm in alive_names]
if len(elim_indices) == 0:
    continue

# 存活者集合（当周活着但不在淘汰列表）
survivor_indices = [i for i in range(len(alive_names)) if i
not in elim_indices]
if len(survivor_indices) == 0:
    continue

for ei in elim_indices:
    #  $T_e \leq T_j$  for all survivors  $j$ 
    constraints += [T[ei] <= T[survivor_indices] -
eps_margin]

# ---- 2.3 跨周平滑项：相邻周都活着的同一选手，p 尽量接近 ----
# 注意：不同周变量维度不同，需要对齐同一选手在各自周内的索引
for idx_w in range(1, len(valid_weeks)):
    w_prev = valid_weeks[idx_w - 1]
    w_curr = valid_weeks[idx_w]
    prev_names = week_alive_names[w_prev]
    curr_names = week_alive_names[w_curr]

    # 交集：两周都在赛的选手
    common =
sorted(set(prev_names).intersection(set(curr_names)))
    if len(common) == 0:
        continue

    p_prev = p_vars[w_prev]
    p_curr = p_vars[w_curr]

    # 构造差分向量
    diffs = []
    for nm in common:
        ip = prev_names.index(nm)
        ic = curr_names.index(nm)

```

```

        diffs.append(p_curr[ic] - p_prev[ip])

    diffs = cp.hstack(diffs)
    obj_terms.append(lam_smooth * smooth_weight *
cp.sum_squares(diffs))

# ---- 2.4 求解凸优化 ----
objective = cp.Minimize(cp.sum(obj_terms))
problem = cp.Problem(objective, constraints)

# 尝试多种求解器 (不同环境可用的求解器不同)
solved = False
last_err = None
for solver in ["ECOS", "SCS", "OSQP"]:
    try:
        problem.solve(solver=solver, verbose=False)
        if problem.status in ["optimal", "optimal_inaccurate"]:
            solved = True
            break
    except Exception as e:
        last_err = e

if not solved:
    raise RuntimeError(f"赛季 {season_id} 优化未能求解成功。最后错误: {last_err}")

# ---- 2.5 整理输出: 投票占比、评委百分比、合成值 ----
rows = []
for w in valid_weeks:
    alive_names = week_alive_names[w]
    J = week_J[w]
    J_sum = float(np.sum(J)) if np.sum(J) > 0 else 1.0
    q = J / J_sum
    p_hat = np.maximum(p_vars[w].value, 0)
    p_hat = p_hat / (p_hat.sum() if p_hat.sum() > 0 else 1.0)
    T_hat = q + p_hat

    for k, nm in enumerate(alive_names):
        rows.append({
            "season": season_id,
            "week": w,
            "contestant": nm,
            "J": float(J[k]),
            "q_hat": float(q[k]),

```

```

        "p_hat": float(p_hat[k]),
        "T_hat": float(T_hat[k]),
    })

votes_long = pd.DataFrame(rows)

# ---- 2.6 不确定性区间: 在“淘汰一致性约束 + 单纯形约束”下做
min/max ----
# 这里为可运行性与速度考虑: 仅做“逐周、逐选手”的区间 (不加跨
周平滑), 更贴近题意的“可辨识区间”
# 如果要加平滑一起做区间, 计算量会非常大 (每个变量都要两次大规
模优化)。
uncert_rows = []
for w in valid_weeks:
    alive_names = week_alive_names[w]
    m = len(alive_names)
    if m <= 1:
        continue

    eliminated_names = elim_map.get(w, [])
    J = week_J[w]
    J_sum = float(np.sum(J)) if np.sum(J) > 0 else 1.0
    q = J / J_sum

    # 构造该周的可行域:  $p \geq 0$ ,  $\sum p = 1$ ,  $(q+p)$  满足淘汰一致性
    p = cp.Variable(m, nonneg=True)
    cons = [cp.sum(p) == 1]
    T = q + p

    elim_indices = [alive_names.index(nm) for nm in
eliminated_names if nm in alive_names]
    if len(elim_indices) > 0:
        survivor_indices = [i for i in range(m) if i not in
elim_indices]
        if len(survivor_indices) > 0:
            for ei in elim_indices:
                cons += [T[ei] <= T[survivor_indices] -
eps_margin]

    # 对每个选手  $i$ : 分别求  $p_i$  的最小值和最大值
    for i in range(m):
        # min
        prob_min = cp.Problem(cp.Minimize(p[i]), cons)
        prob_max = cp.Problem(cp.Maximize(p[i]), cons)

```

```

pmin = np.nan
pmax = np.nan

# 求解 min
ok_min = False
for solver in ["ECOS", "SCS", "OSQP"]:
    try:
        prob_min.solve(solver=solver, verbose=False)
        if prob_min.status in ["optimal",
"optimal_inaccurate"]:
            pmin = float(p.value[i])
            ok_min = True
            break
    except Exception:
        pass

# 求解 max
ok_max = False
for solver in ["ECOS", "SCS", "OSQP"]:
    try:
        prob_max.solve(solver=solver, verbose=False)
        if prob_max.status in ["optimal",
"optimal_inaccurate"]:
            pmax = float(p.value[i])
            ok_max = True
            break
    except Exception:
        pass

if (not ok_min) or (not ok_max):
    # 有些周可能因数据特殊（例如多淘汰/无淘汰/极端分
    值）导致区间问题不稳定
    # 这里记为 NaN，不影响主估计
    uncert_rows.append({
        "season": season_id,
        "week": w,
        "contestant": alive_names[i],
        "p_min": np.nan,
        "p_max": np.nan,
        "width": np.nan,
    })
else:
    width = max(0.0, pmax - pmin)
    uncert_rows.append({

```

```

        "season": season_id,
        "week": w,
        "contestant": alive_names[i],
        "p_min": pmin,
        "p_max": pmax,
        "width": width,
    })

    uncert_long = pd.DataFrame(uncert_rows)
    return votes_long, uncert_long

# ===== 3. 对所有赛季运行（并输出结果表 + 图片）
=====
all_votes = []
all_uncert = []

seasons_sorted = sorted(season_weeks.keys())

for s in seasons_sorted:
    g = df[df[season_col] == s].copy()
    valid_ws = season_weeks[int(s)]
    if len(valid_ws) < 2:
        continue

    elim_map = infer_eliminations_one_season(g, valid_ws)

    # 规则提示（但我们在代码里统一用“百分比合并淘汰约束”近似，保证可跑与稳定）
    rule = season_rule(int(s))
    if rule == "rank_like":
        warnings.warn(
            f"赛季 {s} 可能属于“按排名合并”的时期。为保证可运行性，本代码仍用“百分比合并淘汰约束”近似反推投票。"
        )

    votes_long, uncert_long =
solve_votes_one_season_percent_approx(
    g_season=g,
    valid_weeks=valid_ws,
    elim_map=elim_map,
    lam_entropy=0.60,
    lam_smooth=0.40,
    smooth_weight=5.0,

```

```

        eps_margin=1e-6
    )

    all_votes.append(votes_long)
    all_uncert.append(uncert_long)

# 合并全赛季结果
votes_all = pd.concat(all_votes, ignore_index=True) if
len(all_votes) else pd.DataFrame()
uncert_all = pd.concat(all_uncert, ignore_index=True) if
len(all_uncert) else pd.DataFrame()

# ---- 3.1 输出估计投票（占比 & 相对票数） ----
# 题面示例用“总票数=1000 万”这类尺度，我们也给出一个可读的相对票
# 数列（可改）
V_TOT = 10_000_000
votes_all["V_hat"] = votes_all["p_hat"] * V_TOT

votes_csv = os.path.join(RESULT_DIR, "Q1_estimated_votes_long.csv")
votes_all.to_csv(votes_csv, index=False, encoding="utf-8-sig")

uncert_csv = os.path.join(RESULT_DIR,
"Q1_vote_uncertainty_intervals.csv")
uncert_all.to_csv(uncert_csv, index=False, encoding="utf-8-sig")

# ===== 4. 可视化（保存到 result1，不展示） =====
def save_season_heatmap(votes_long: pd.DataFrame, season_id: int,
top_n: int = 18):
    """
    画一个赛季的“投票占比热力图”：x=周次，y=选手（取累计投票占比
    最高的 top_n），颜色=p_hat
    """
    season_df = votes_long[votes_long["season"] ==
season_id].copy()
    if season_df.empty:
        return

    # 选 top_n: 按整季 p_hat 累计
    total =
season_df.groupby("contestant")["p_hat"].sum().sort_values(ascendin
g=False)
    keep = total.head(top_n).index.tolist()
    season_df = season_df[season_df["contestant"].isin(keep)]

```

```

    pivot = season_df.pivot_table(index="contestant",
columns="week", values="p_hat", fill_value=0.0)
    # 排序: 累计高的在上面
    pivot = pivot.loc[keep]

    fig, ax = plt.subplots(figsize=(12, 6), dpi=200)
    im = ax.imshow(pivot.values, aspect="auto",
interpolation="nearest")

    ax.set_title(f"Season {season_id}: 粉丝投票占比估计 (Top
{top_n} 选手热力图)", fontsize=13, pad=12)
    ax.set_xlabel("Week")
    ax.set_ylabel("Contestant")

    ax.set_xticks(np.arange(pivot.shape[1]))
    ax.set_xticklabels(pivot.columns.tolist(), rotation=0,
fontsize=8)

    ax.set_yticks(np.arange(pivot.shape[0]))
    ax.set_yticklabels(pivot.index.tolist(), fontsize=8)

    cbar = fig.colorbar(im, ax=ax, fraction=0.025, pad=0.02)
    cbar.set_label("Estimated vote share (p_hat)", rotation=90)

    fig.tight_layout()
    out = os.path.join(RESULT_DIR,
f"season_{season_id}_vote_share_heatmap.png")
    fig.savefig(out, bbox_inches="tight")
    plt.close(fig)

def save_uncertainty_plot(uncert_long: pd.DataFrame, season_id:
int):
    """
    画一个赛季的“不确定性宽度”分布:
    - 每周平均 width
    - 同时画每周的箱线图 (如果数据足够)
    """
    season_u = uncert_long[uncert_long["season"] ==
season_id].copy()
    season_u = season_u.dropna(subset=["width"])
    if season_u.empty:
        return

```

```

# 每周平均不确定性
wmean = season_u.groupby("week")["width"].mean().sort_index()

fig, ax = plt.subplots(figsize=(12, 5), dpi=200)
ax.plot(wmean.index, wmean.values, marker="o", linewidth=2)

ax.set_title(f"Season {season_id}: 投票估计不确定性（可行区间宽度）周平均", fontsize=13, pad=12)
ax.set_xlabel("Week")
ax.set_ylabel("Mean interval width")

fig.tight_layout()
out = os.path.join(RESULT_DIR,
f"season_{season_id}_uncertainty_mean_by_week.png")
fig.savefig(out, bbox_inches="tight")
plt.close(fig)

# 箱线图（每周选手宽度分布）
weeks = sorted(season_u["week"].unique().tolist())
data = [season_u.loc[season_u["week"] == w, "width"].values for
w in weeks]

fig2, ax2 = plt.subplots(figsize=(13, 5), dpi=200)
ax2.boxplot(data, labels=weeks, showfliers=False)
ax2.set_title(f"Season {season_id}: 投票估计不确定性（各周选手区间宽度箱线图）", fontsize=13, pad=12)
ax2.set_xlabel("Week")
ax2.set_ylabel("Interval width")
ax2.tick_params(axis="x", labelrotation=0)

fig2.tight_layout()
out2 = os.path.join(RESULT_DIR,
f"season_{season_id}_uncertainty_boxplot.png")
fig2.savefig(out2, bbox_inches="tight")
plt.close(fig2)

def save_elimination_check_plot(votes_long: pd.DataFrame,
season_id: int, elim_map: Dict[int, List[str]]):
    """
    画“淘汰一致性检查图”：
    - 对每周，把合成值  $T_{\hat{}}$  从小到大排序
    - 标记真实淘汰者在该排序中的位置（越靠左越符合“最低淘汰”）

```

```

"""
    season_df = votes_long[votes_long["season"] ==
season_id].copy()
    if season_df.empty:
        return

    weeks = sorted(season_df["week"].unique().tolist())
    # 统计每周淘汰者在排序中的名次 (1=最差)
    rank_rows = []
    for w in weeks:
        wdf = season_df[season_df["week"] == w].copy()
        wdf = wdf.sort_values("T_hat", ascending=True) # T_hat 越
小越危险
        wdf["rank_T"] = np.arange(1, len(wdf) + 1)

        elims = elim_map.get(w, [])
        if len(elims) == 0:
            continue
        for e in elims:
            if e in wdf["contestant"].values:
                rk = int(wdf.loc[wdf["contestant"] == e,
"rank_T"].iloc[0])
                rank_rows.append({"week": w, "contestant": e,
"rank_T": rk, "n_alive": len(wdf)})

    if len(rank_rows) == 0:
        return

    r = pd.DataFrame(rank_rows)
    r["rank_frac"] = r["rank_T"] / r["n_alive"]

    fig, ax = plt.subplots(figsize=(12, 5), dpi=200)
    ax.scatter(r["week"], r["rank_frac"], s=35)

    ax.set_title(f"Season {season_id}: 真实淘汰者在合成排序中的位置
(越接近 0 越符合最低淘汰)", fontsize=13, pad=12)
    ax.set_xlabel("Week")
    ax.set_ylabel("Eliminated rank / alive count")

    ax.set_ylim(-0.02, 1.02)
    fig.tight_layout()
    out = os.path.join(RESULT_DIR,
f"season_{season_id}_elimination_rank_check.png")
    fig.savefig(out, bbox_inches="tight")

```

```

plt.close(fig)

# ---- 4.1 选择若干赛季做图（默认：第 3、11、27、28、34 季如果存在；
否则取前 3 个赛季） ----
preferred = [3, 11, 27, 28, 34]
available = sorted(votes_all["season"].unique().tolist()) if not
votes_all.empty else []
plot_seasons = [s for s in preferred if s in available]
if len(plot_seasons) == 0:
    plot_seasons = available[:3]

for s in plot_seasons:
    # 为淘汰检查图，需要 elim_map，因此重新推一次（很快）
    g = df[df[season_col] == s].copy()
    valid_ws = season_weeks[int(s)]
    elim_map = infer_eliminations_one_season(g, valid_ws)

    save_season_heatmap(votes_all, season_id=int(s), top_n=18)
    save_uncertainty_plot(uncert_all, season_id=int(s))
    save_elimination_check_plot(votes_all, season_id=int(s),
elim_map=elim_map)

# ===== 5. 额外输出：快速摘要（便于你写报告）
=====
# 计算整体淘汰一致性：真实淘汰者在  $T_{hat}$  排序中是否为第 1（最差）
def compute_elimination_hit_rate(votes_all_df: pd.DataFrame) ->
pd.DataFrame:
    out_rows = []
    for s in sorted(votes_all_df["season"].unique()):
        g = df[df[season_col] == s].copy()
        valid_ws = season_weeks[int(s)]
        elim_map = infer_eliminations_one_season(g, valid_ws)

        season_df = votes_all_df[votes_all_df["season"] ==
s].copy()
        if season_df.empty:
            continue

        hits = 0
        total = 0
        for w in sorted(season_df["week"].unique()):
            elims = elim_map.get(w, [])

```

```

        if len(elims) == 0:
            continue

        wdf = season_df[season_df["week"] == w].copy()
        wdf = wdf.sort_values("T_hat", ascending=True) # T_hat
        越小越危险

        if wdf.empty:
            continue

        # 真实淘汰者可能多个：只要都在“最危险的若干位”就算命
        中

        # 这里简单：若淘汰 1 人则看是否最小；若淘汰 k 人则看是否
        都在前 k

        k = len([e for e in elims if e in
wdf["contestant"].values])
        if k == 0:
            continue

        total += 1
        most_dangerous =
set(wdf.head(k)["contestant"].tolist())
        if set(elims).issubset(most_dangerous):
            hits += 1

        hit_rate = hits / total if total > 0 else np.nan
        out_rows.append({"season": int(s), "weeks_with_elim":
total, "hit_rate": hit_rate})
        return pd.DataFrame(out_rows)

summary_df = compute_elimination_hit_rate(votes_all)
summary_csv = os.path.join(RESULT_DIR,
"Q1_elimination_consistency_summary.csv")
summary_df.to_csv(summary_csv, index=False, encoding="utf-8-sig")

print("运行完成 ☑")
print(f"- 估计投票结果: {votes_csv}")
print(f"- 不确定性区间: {uncert_csv}")
print(f"- 淘汰一致性摘要: {summary_csv}")
print(f"- 图片输出目录: {RESULT_DIR}")

```

问题 2

问题 2 分析

1) 思路框架：用“反事实回放”把两套规则放在同一条赛程里公平对比，并专门检验争议人物与“评委救场”机制的抑制能力

第 2 问的任务不是再去估计投票，而是把第 1 问得到的“粉丝投票估计值”与原始数据中的评委评分结合起来，做一件更贴合节目制度评估的事：在同一批参赛者、同一周的评委评分不变的前提下，分别用“按排名合并”和“按百分比合并”两种规则回放整季淘汰路径，观察它们对淘汰顺序、最终名次以及“评委低分但人气极强”的争议人物的影响差异。题面给出的背景非常明确：第 1、2 季采用按排名合并，因第 2 季 Jerry Rice 在多周评委最低却仍进入决赛而改为按百分比合并；第 27 季 Bobby Bones 低评委分却夺冠后，从第 28 季开始引入“先合成确定倒数两名，再由评委在直播中投票淘汰其中一位”的轻微调整，并且大约同一时期制作方又恢复了早期的按排名合并（确切季数不明，但题面认为第 28 季是合理推测）。这意味着第 2 问的比较必须同时覆盖三层制度变量：两种合成方式本身、合成方式对“观众倾向/评委倾向”的偏置、以及“评委从倒二中选择淘汰者”的二次裁决机制对争议结果的干预强度

在操作层面，本题最关键的“对比公平性”不是靠主观讨论，而是用同一套投票估计 $\hat{p}_{i,t}$

（投票占比）在两套规则下分别生成每周淘汰者与最终名次，再用量化指标回答三类问题：第一类是“整体差异有多大”，例如每周淘汰人是否一致、最终名次的排序相关性如何；第二类是“是否更偏观众”，即当两套规则产生分歧时，哪套规则更容易让评委低分选手存活、更容易让评委高分选手被淘汰；第三类是“争议案例会不会被同样放大”，也就是把 Jerry Rice、Billy Ray Cyrus、Bristol Palin、Bobby Bones 这些题面指定案例放到同一套反事实框架里检验：若换用另一种合成规则，或再叠加“倒二评委投票淘汰”的机制，他们的轨迹是否会发生系统性改变。这样写出来的结论会非常“贴赛题”，因为你不是泛泛而谈“百分比更公平”，而是在“同一赛程回放”的证据下说明哪种制度更可能产生类似第 2 季和第 27 季的结果，以及评委救场机制在多大程度上能抑制这种结果。

解题思路：

2) 数学模型构建：两套合成规则的统一表达、偏置度量与“评委救场”的可调机制

2.1 赛季—周一选手结构与投票估计的接入方式（对齐题面数据口径）

沿用数据文件结构，对每个赛季 s 、每一周 t 、仍在赛的参赛者集合记为 $\mathcal{A}_{s,t}$ 。评委

打分按字段 `weekX_judgeY_score` 汇总为当周评委总分 $J_{i,t}$ ，并进一步形成评委百分比

$q_{i,t}$ ：

$$J_{i,t} = \sum_{y \in \mathcal{Y}_{s,t}} x_{i,t,y}, \quad q_{i,t} = \frac{J_{i,t}}{\sum_{k \in \mathcal{A}_{s,t}} J_{k,t}} \quad (18)$$

第 1 问输出的是每位选手每周粉丝投票估计。为了与附录“百分比合并”的制度完全一致（观众投票占比参与合成），我们以投票占比 $\hat{p}_{i,t}$ 接入第 2 问，并满足

$$\sum_{i \in \mathcal{A}_{s,t}} \hat{p}_{i,t} = 1, \quad \hat{p}_{i,t} \geq 0 \quad (19)$$

这一步的意义是：无论你在第 1 问中把投票估计为“110 万/370 万”还是其他尺度，都可以通过归一化得到 $\hat{p}_{i,t}$ ，从而保证不同赛季、不同周次之间具有可比性，并且直接适配题面给出的“评委百分比 + 观众百分比”的合成方式示例。

2.2 两种合成方法的统一描述：从“淘汰函数”到“整季回放”

(1) 按百分比合并（第 3 季至第 27a 季的典型做法）

按题面附录示例，合成得分为评委百分比与观众投票百分比之和，得分最低者淘汰。对应到符号可写为 $T^{(P)}i,t = q_{i,t} + \hat{p}_{i,t}$ ，并定义当周淘汰算子 $\mathcal{E}^{(P)}(s,t)$ ：

$$T_{i,t}^{(P)} = q_{i,t} + \hat{p}_{i,t}, \quad \mathcal{E}^{(P)}(s,t) = \arg \min_{i \in \mathcal{A}_{s,t}} T_{i,t}^{(P)} \quad (20)$$

若出现“多淘汰周”，则将 $\arg \min$ 推广为取最小的若干个即可。

(2) 按排名合并（第 1、2 季及第 28a-34 季的典型做法）

题面明确“按排名合并”是分别对评委总分与观众投票做排名，再将名次相加，排名之和最差者淘汰。因此令评委名次 $r_{i,t}^J = \text{rank} \downarrow (J_{i,t})$ （分高名次小），观众名次 $r_{i,t}^V = \text{rank} \downarrow (\hat{p}_{i,t})$ （票高名次小），合成名次和为 $S_{i,t} = r_{i,t}^J + r_{i,t}^V$ ，淘汰算子为

$$S_{i,t} = r_{i,t}^J + r_{i,t}^V, \quad \mathcal{E}^{(R)}(s,t) = \arg \max_{i \in \mathcal{A}_{s,t}} S_{i,t} \quad (21)$$

(3) 整季回放的递推结构（把“每周淘汰”串成“最终名次”）

无论规则为百分比或排名，都可以用同一个递推框架把周周淘汰连成整季名次：令 $\mathcal{A}_{s,1}$

为开赛名单，若规则记为 $g \in P, R$ ，则

$$e_{s,t}^{(g)} = \mathcal{E}^{(g)}(s,t), \quad \mathcal{A}_{s,t+1}^{(g)} = \mathcal{A}_{s,t}^{(g)} \setminus \{e_{s,t}^{(g)}\} \quad (22)$$

当周次推进到决赛周时，按照题面“决赛周对剩余三对（或更多）进行综合排名”的描述，用对应规则对决赛周的 $T^{(P)}i,t$ 或 $S_{i,t}$ 排序即可得到最终名次序列 $\pi_s^{(g)}$ 。

2.3 “更偏观众还是更偏评委”的量化：把偏置写成可比较的指标，而不是口头结论

题面第 2 问明确要求：若两种方法产生差异，是否有一种更倾向重视观众评分而忽略评委评分。因此我们需要构造“偏观众”的度量，并且必须能在两套规则下同时计算。一个与赛题高度契合、解释也直观的做法是把“淘汰发生时的评委位置”当作偏置切片：如果某规则经常淘汰评委高分者，说明观众权重过强；如果经常淘汰评委低分者，说明更偏评委。令当周被淘汰者为 $e_{s,t}^{(g)}$ ，其评委名次为 $r_{e,t}^J$ ，则定义“评委保护强度”（越大越保护评委高分）：

$$B^{(g)} = \frac{1}{N} \sum_{s,t} \left(1 - \frac{r_{e_{s,t}^{(g)}}^J - 1}{|\mathcal{A}_{s,t}| - 1} \right) \quad (23)$$

其中 N 是所有发生淘汰的周数总和。直观上，如果淘汰者经常是评委最低分（ r^J 大），则括号内数值小， $B^{(g)}$ 低；如果淘汰者经常不是评委低分，说明观众力量把评委高分者“拉下去”， $B^{(g)}$ 会更高或更低取决于你的方向约定。为了避免方向混乱，也可以直接用“淘汰者的评委分位数”作为偏置报告核心图表：

$$\phi_{s,t}^{(g)} = \frac{r_{e_{s,t}^{(g)}}^J - 1}{|\mathcal{A}_{s,t}| - 1} \in [0,1] \quad (24)$$

ϕ 越接近 1 表示淘汰者越接近评委最差，制度越“尊重评委”； ϕ 越接近 0 表示淘汰者越接近评委最好，制度越“被观众翻盘”。这类定义与题面“评委评分意在体现技术、粉丝投票更受知名度影响”这一背景叙述天然吻合

2.4 “评委救场”机制的建模：把第 28 季后的流程写成可回放的二阶段决策

题面描述的第 28 季起调整是：先用评委评分与观众投票合成确定倒数两名，然后评委在直播中投票决定淘汰哪一位。这可以建成一个二阶段淘汰算子：第一阶段产生倒二集合 $\mathcal{B}_{s,t}^{(g)}$ ，第二阶段由评委在倒二中做最终淘汰选择。

第一阶段对任意规则 g ，定义倒二集合为

$$\mathcal{B}_{s,t}^{(g)} = \text{Bottom } 2 \left(\left\{ T_{i,t}^{(P)} \right\}_{i \in \mathcal{A}_{s,t}} \right) \text{ 或 } \mathcal{B}_{s,t}^{(g)} = \text{Top } 2 \left(\left\{ S_{i,t} \right\}_{i \in \mathcal{A}_{s,t}} \right) \quad (25)$$

其中 Bottom2 表示取合成得分最小的两人，Top2 表示取名次和最大的两人。

第二阶段需要将“评委投票”转化为可计算规则。由于数据中没有评委投票结果，最贴合题意且可解释的代理是：评委更可能淘汰“技术更差”的那一位，即在倒二中评委得分更低者被淘汰概率更高。用 $J_{i,t}$ 作为评委对技术的综合判断，构造一个带参数的选择模型：

$$\mathbb{P} \left(e_{s,t}^{(JS,g)} = a \mid \mathcal{B}_{s,t}^{(g)} = \{a, b\} \right) = \frac{\exp(-\kappa J_{a,t})}{\exp(-\kappa J_{a,t}) + \exp(-\kappa J_{b,t})} \quad (26)$$

其中 $\kappa \geq 0$ 表示评委“救场力度”：当 $\kappa \rightarrow \infty$ 时，评委几乎必然淘汰倒二里评委分更低者；当 $\kappa = 0$ 时，评委相当于随机选。这个建模方式的价值在于你可以用 κ 做敏感性分析：评委救场到底需要多强，才能显著降低“低评委分但高人气”的争议结果出现概率，从而回应题面最后那句“是否建议加入让评委从排名最后的两对中选择”的制度建议问题。

3) 模型算法求解：按赛季回放、按案例对照、按指标输出推荐结论

3.1 赛季回放引擎：同一套投票估计下的两规则淘汰路径对齐

算法层面最核心的是构建一个“赛季回放引擎”，它以赛季 s 为单位，输入第 1 问的投票占比估计 $\hat{p}_{i,t}$ 与数据中的 $J_{i,t}$ ，输出两套规则下的淘汰序列 $e_{s,t}^{(P)}$ 、 $e_{s,t}^{(R)}$ 与最终名次

$\pi_s^{(P)}$ 、 $\pi_s^{(R)}$ 。回放过程严格按周递推：每一周只在仍在赛集合 $\mathcal{A}_{s,t}^{(g)}$ 内计算合成得分并

淘汰，直到进入决赛周按合成得分排序产生 1–3 名（或更多名次）。这一设计与题面“每周淘汰、决赛周排名”的节目结构一致，且能自然处理“某周无淘汰或多淘汰”的数据特性。

为了把两套规则的差异定量化，直接统计“周淘汰一致率”和“最终名次一致性”。例如周淘汰一致率可定义为

$$\text{WeekAgree} = \frac{1}{N} \sum_{s,t} \mathbf{1}(e_{s,t}^{(P)} = e_{s,t}^{(R)}) \quad (27)$$

最终名次一致性可以用名次相关 (Spearman) 或 Kendall 距离衡量，你在正文中用文字解释它表示“整季排序差异的整体程度”，即可避免陷入纯公式堆砌。

3.2 偏置与分歧解释：用“评委分位数分布”回答“更偏观众吗”

在得到两套规则的淘汰序列后，计算每次淘汰对应的评委分位数 $\phi_{s,t}^{(g)}$ ，并比较两套规则

下 ϕ 的分布差异。如果发现某规则下大量淘汰发生在 ϕ 较小（淘汰者并非评委低分）的位置，就可以非常直接地解释为“该规则更容易出现观众力量逆转评委意见”的现象，这恰恰对应题面所说“评委评价技术、粉丝投票受知名度影响，制作方甚至可能希望分歧以提升热度”的背景逻辑。此时你给出的不是抽象观点，而是：在历史数据与同一套投票估计接入下，哪种规则产生的“评委高分被淘汰”频率更高，从而回答“是否更倾向重视观众评分而忽略评委评分”的问题。

3.3 争议案例的反事实检验：把题面四个例子当作“制度压力测试样本”

题面直接点名了第 2 季 Jerry Rice、第 4 季 Billy Ray Cyrus、第 11 季 Bristol Palin、第 27 季 Bobby Bones 作为评委与粉丝意见分歧的争议案例，并要求你考察两种合成方式是否对他们产生相同影响。因此在回放引擎的输出基础上，对每个案例选手 c ，抽取其在真实赛季中每周的 $(J_{c,t}, \hat{p}_{c,t})$ 与对应淘汰周 t^* ，然后在两套规则下计算“若该规则贯穿整季，他会在第几周出局/最终名次是什么”。形式上可把“生存周数”定义为

$$\text{SurvivalWeeks}^{(g)}(c) = \max \{t : c \in \mathcal{A}_{s,t}^{(g)}\} \quad (28)$$

如果某位争议选手在规则 g_1 下能显著延长生存周数、甚至夺冠，而在 g_2 下明显提前出局，就可以把这种差异归因于制度对“高人气低技术”的放大能力不同。这一段写出来会非常贴题，因为你在回答“这些参赛者的结果是否会受到相同影响”，而不是泛泛地评价规则优劣。

Python 代码：

```
# -*- coding: utf-8 -*-
"""
```

问题 2: 比较“按百分比合并” vs “按排名合并” 两种规则对淘汰路径/最终名次的影响,

并评估“倒二评委救场 (Judge Save)”机制对争议结果的抑制效果 (参数化 κ)。

重要说明 (面向快速跑通)：

- 代码会优先读取 `result1/Q1_estimated_votes_long.csv` (来自问题 1 的投票估计结果)。
 - 如果该文件不存在, 则会自动在本脚本内“快速近似反推投票占比 p_{hat} ”
(使用百分比合并淘汰一致性约束 + 最大熵 + 平滑的凸优化, 与问题 1 的代码一致)。
 - 所有图片保存到 `result1` 文件夹, 不展示。
 - 数据文件就在根目录: `2026_MCM_Problem_C_Data.csv`
- """

```
import os
import re
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 若需自动反推投票, 会用到 cvxpy
# pip install cvxpy
import cvxpy as cp

# =====
# 0. 输出目录与绘图风格
# =====
RESULT_DIR = "result2"
os.makedirs(RESULT_DIR, exist_ok=True)

plt.style.use("seaborn-v0_8-whitegrid")
np.random.seed(42)

# =====
# 1. 读取真实数据并解析结构
# =====
DATA_FILE = "2026_MCM_Problem_C_Data.csv"
df = pd.read_csv(DATA_FILE)

def find_col(candidates, columns):
    """在列名中尽量稳健地找到目标列"""
```

```

for c in candidates:
    if c in columns:
        return c
cols_lower = {x.lower(): x for x in columns}
for c in candidates:
    if c.lower() in cols_lower:
        return cols_lower[c.lower()]
raise ValueError(f"找不到列: 候选={candidates}")

cols = df.columns.tolist()
season_col = find_col(["season", "Season", "SEASON"], cols)

# 尝试找选手标识列 (名人姓名列)
try:
    contestant_col = find_col(
        ["celebrity", "Celebrity", "contestant", "Contestant",
        "star", "Star", "name", "Name"],
        cols
    )
except Exception:
    obj_cols = [c for c in cols if df[c].dtype == "object"]
    if not obj_cols:
        raise ValueError("数据中没有明显文本列可作为选手标识。请检查 CSV。")
    contestant_col = sorted(obj_cols, key=lambda c:
df[c].nunique(), reverse=True)[0]
    warnings.warn(f"未识别到标准选手列名, 暂用 {contestant_col} 作为选手标识列")

# 解析 weekX_judgeY_score 列
week_judge_pattern = re.compile(r"week(\d+)_judge(\d+)_score",
re.IGNORECASE)
week_cols = {}
for c in cols:
    m = week_judge_pattern.match(c)
    if m:
        w = int(m.group(1))
        week_cols.setdefault(w, []).append(c)

if not week_cols:
    raise ValueError("未找到 weekX_judgeY_score 列, 请检查 CSV 格式。")

all_weeks = sorted(week_cols.keys())

```

```

# 计算每周评委总分  $J_{week\{w\}}$ 
for w in all_weeks:
    df[f"J_week{w}"] = df[week_cols[w]].apply(pd.to_numeric,
errors="coerce").sum(axis=1, skipna=True)

# 每赛季有效周: 该周至少有一人  $J>0$ 
season_weeks = {}
for s, g in df.groupby(season_col):
    valid_ws = []
    for w in all_weeks:
        if (g[f"J_week{w}"] > 0).any():
            valid_ws.append(w)
    season_weeks[int(s)] = valid_ws

# =====
# 2. 工具函数: 推断淘汰者 (用 0 分机制)
# =====
def infer_eliminations_one_season(g_season: pd.DataFrame,
valid_weeks):
    """
    根据: 本周  $J>0$  且 下周  $J==0 \Rightarrow$  本周淘汰
    返回 week  $\rightarrow$  [淘汰者名单] (可能为空/多个)
    """
    elim_map = {}
    names = g_season[contestant_col].astype(str).values

    for idx in range(len(valid_weeks) - 1):
        w = valid_weeks[idx]
        w_next = valid_weeks[idx + 1]
        alive_w = g_season[f"J_week{w}"].values > 0
        alive_next = g_season[f"J_week{w_next}"].values > 0
        eliminated = names[(alive_w) & (~alive_next)]
        elim_map[w] = list(eliminated)

    elim_map[valid_weeks[-1]] = []
    return elim_map

# =====
# 3. 如果 Q1 投票估计不存在: 快速反推  $p_{hat}$  (用于 Q2 回放)
# (与问题 1 一致, 采用“百分比淘汰约束”近似, 保证能跑)
# =====

```

```

def solve_votes_one_season_percent_approx(
    g_season: pd.DataFrame,
    valid_weeks,
    elim_map,
    lam_entropy=0.60,
    lam_smooth=0.40,
    smooth_weight=5.0,
    eps_margin=1e-6,
):
    """
    估计每周投票占比  $p_{hat}$  (当周存活选手之和为1)
    约束:  $T=q+p$ , 真实淘汰者  $T$  最小 (允许多淘汰)
    目标: 最大熵 (最少偏置) + 跨周平滑 (投票不乱跳)
    返回: 长表  $votes\_long$ :  $season, week, contestant, J, q\_hat, p\_hat, T\_hat$ 
    """

    season_id = int(g_season[season_col].iloc[0])
    contestants = g_season[contestant_col].astype(str).tolist()
    n_total = len(contestants)

    # 每周活着的选手与 J
    week_alive_names = {}
    week_J = {}
    for w in valid_weeks:
        alive_mask = g_season[f"J_week{w}"].values > 0
        alive_names = [contestants[i] for i in range(n_total) if
            alive_mask[i]]
        week_alive_names[w] = alive_names
        week_J[w] = g_season.loc[alive_mask,
            f"J_week{w}"].values.astype(float)

    # 建变量与约束
    p_vars = {}
    constraints = []
    obj_terms = []

    for w in valid_weeks:
        m = len(week_alive_names[w])
        p = cp.Variable(m, nonneg=True)
        p_vars[w] = p
        constraints.append(cp.sum(p) == 1)
        # 最大熵: 最小化  $\sum p \log p = -\sum \text{entr}(p)$ 
        obj_terms.append(lam_entropy * cp.sum(-cp.entr(p)))

```

```

# 淘汰一致性约束:  $T=q+p$ , 淘汰者  $T \leq$  存活者  $T$ 
for w in valid_weeks:
    elims = elim_map.get(w, [])
    if not elims:
        continue
    alive_names = week_alive_names[w]
    m = len(alive_names)
    if m <= 1:
        continue

    J = week_J[w]
    J_sum = float(np.sum(J)) if np.sum(J) > 0 else 1.0
    q = J / J_sum

    p = p_vars[w]
    T = q + p

    elim_idx = [alive_names.index(nm) for nm in elims if nm in
alive_names]
    if not elim_idx:
        continue
    surv_idx = [i for i in range(m) if i not in elim_idx]
    if not surv_idx:
        continue

    for ei in elim_idx:
        constraints.append(T[ei] <= T[surv_idx] - eps_margin)

# 跨周平滑: 相邻两周都在赛的选手  $p$  尽量接近
for i in range(1, len(valid_weeks)):
    w_prev = valid_weeks[i - 1]
    w_curr = valid_weeks[i]
    prev_names = week_alive_names[w_prev]
    curr_names = week_alive_names[w_curr]
    common = sorted(set(prev_names).intersection(curr_names))
    if not common:
        continue
    diffs = []
    for nm in common:
        ip = prev_names.index(nm)
        ic = curr_names.index(nm)
        diffs.append(p_vars[w_curr][ic] - p_vars[w_prev][ip])
    diffs = cp.hstack(diffs)
    obj_terms.append(lam_smooth * smooth_weight *

```

```

cp.sum_squares(diffs))

    problem = cp.Problem(cp.Minimize(cp.sum(obj_terms)),
constraints)

    solved = False
    last_err = None
    for solver in ["ECOS", "SCS", "OSQP"]:
        try:
            problem.solve(solver=solver, verbose=False)
            if problem.status in ["optimal", "optimal_inaccurate"]:
                solved = True
                break
        except Exception as e:
            last_err = e

    if not solved:
        raise RuntimeError(f"赛季{season_id} 投票估计失败:
{last_err}")

    # 整理输出
    rows = []
    for w in valid_weeks:
        alive_names = week_alive_names[w]
        J = week_J[w]
        J_sum = float(np.sum(J)) if np.sum(J) > 0 else 1.0
        q = J / J_sum
        p_hat = np.maximum(p_vars[w].value, 0)
        p_hat = p_hat / (p_hat.sum() if p_hat.sum() > 0 else 1.0)
        T_hat = q + p_hat

        for k, nm in enumerate(alive_names):
            rows.append({
                "season": season_id,
                "week": int(w),
                "contestant": nm,
                "J": float(J[k]),
                "q_hat": float(q[k]),
                "p_hat": float(p_hat[k]),
                "T_hat": float(T_hat[k]),
            })

    return pd.DataFrame(rows)

```

```

# =====
# 4. 载入（或生成）Q1 投票估计结果 votes_all
# =====
Q1_FILE = os.path.join(RESULT_DIR, "Q1_estimated_votes_long.csv")

if os.path.exists(Q1_FILE):
    votes_all = pd.read_csv(Q1_FILE)
    # 统一列名（防止编码/大小写差异）
    rename_map = {}
    for c in votes_all.columns:
        if c.lower() == "contestant":
            rename_map[c] = "contestant"
        if c.lower() == "season":
            rename_map[c] = "season"
        if c.lower() == "week":
            rename_map[c] = "week"
        if c.lower() == "p_hat":
            rename_map[c] = "p_hat"
        if c.lower() == "q_hat":
            rename_map[c] = "q_hat"
        if c.lower() == "t_hat":
            rename_map[c] = "T_hat"
    votes_all = votes_all.rename(columns=rename_map)
    print(f"已读取问题 1 投票估计: {Q1_FILE}")
else:
    print("未找到问题 1 投票估计文件，将自动估计 p_hat（可能稍慢，但可运行）。")
    all_votes = []
    for s in sorted(season_weeks.keys()):
        g = df[df[season_col] == s].copy()
        valid_ws = season_weeks[int(s)]
        if len(valid_ws) < 2:
            continue
        elim_map = infer_eliminations_one_season(g, valid_ws)
        votes_long = solve_votes_one_season_percent_approx(g,
valid_ws, elim_map)
        all_votes.append(votes_long)
    votes_all = pd.concat(all_votes, ignore_index=True)
    # 保存，方便下次直接用
    votes_all.to_csv(Q1_FILE, index=False, encoding="utf-8-sig")
    print(f"已输出问题 1 投票估计: {Q1_FILE}")

# 确保类型正确

```

```

votes_all["season"] = votes_all["season"].astype(int)
votes_all["week"] = votes_all["week"].astype(int)
votes_all["contestant"] = votes_all["contestant"].astype(str)

# =====
# 5. Q2 核心: 规则回放 (百分比 vs 排名) + Judge Save (倒二评委救场)
# =====
def rank_desc(values: pd.Series, method="min"):
    """数值越大名次越好: rank=1 最好"""
    return (-values).rank(method=method,
ascending=True).astype(int)

def simulate_season(
    season_id: int,
    votes_season: pd.DataFrame,
    elim_map_real: dict,
    rule: str = "percent",
    judge_save: bool = False,
    kappa: float = 50.0,
    rng: np.random.Generator = np.random.default_rng(42),
):
    """
    对单赛季做“规则回放”:
    - rule="percent": 用  $T=q+p$ , 淘汰  $T$  最小
    - rule="rank": 用  $S=rJ+rV$ , 淘汰  $S$  最大
    - judge_save=True: 先找倒二, 再由评委在倒二中“更可能淘汰低  $J$  者” (softmax 参数 kappa)
      * 注意: 这里的“倒二”由该规则的合成分数决定 (percent 用  $T$ , rank 用  $S$ )
    输出:
    - elim_pred: dict week -> [预测淘汰者] (支持多淘汰: 这里默认每周淘汰 1 人, 贴近多数周)
    - final_order: list 最终名次 (冠军在第 1 位)
    - log_df: 每周存活选手的合成分数 (用于分析/画图)
    """
    # 该赛季的有效周 (来自 votes_all)
    weeks = sorted(votes_season["week"].unique().tolist())
    # 初始存活集合: 第一周出现的选手
    alive = set(votes_season[votes_season["week"] == weeks[0]]["contestant"].tolist())

    elim_pred = {}

```

```

logs = []

# 逐周淘汰，直到最后一周（最后一周不淘汰，只排名）
for w in weeks[:-1]:
    wdf = votes_season[votes_season["week"] == w].copy()
    # 仅保留当前存活
    wdf = wdf[wdf["contestant"].isin(alive)].copy()
    if wdf.empty or len(alive) <= 1:
        break

    # 计算本周合成分
    if rule == "percent":
        wdf["score"] = wdf["q_hat"].astype(float) +
wdf["p_hat"].astype(float) # 越小越危险
        wdf = wdf.sort_values("score", ascending=True)
        danger_order = wdf["contestant"].tolist() # 前面最危险
        # 倒二集合
        bottom2 = danger_order[:2] if len(danger_order) >= 2
    else danger_order[:]:
        if not judge_save:
            eliminated = bottom2[0] # 最危险者淘汰
        else:
            # 评委更可能淘汰 J 更低者:  $P(\text{elim}=a) \propto \exp(-\text{kappa} * J_a)$ 
            bdf = wdf[wdf["contestant"].isin(bottom2)].copy()
            Js = bdf.set_index("contestant")["J"].to_dict()
            a, b = bottom2[0], bottom2[1] if len(bottom2) > 1
            else (bottom2[0], bottom2[0])
            wa = np.exp(-kappa * float(Js[a]))
            wb = np.exp(-kappa * float(Js[b]))
            pa = wa / (wa + wb) if (wa + wb) > 0 else 0.5
            eliminated = a if rng.random() < pa else b

    elif rule == "rank":
        # 名次: 大者名次更好, rank=1 最好
        wdf["rJ"] = rank_desc(wdf["J"].astype(float))
        wdf["rV"] = rank_desc(wdf["p_hat"].astype(float))
        wdf["score"] = wdf["rJ"] + wdf["rV"] # 越大越危险 (名
次和越大越差)
        wdf = wdf.sort_values("score", ascending=False)
        danger_order = wdf["contestant"].tolist() # 前面最危险
        bottom2 = danger_order[:2] if len(danger_order) >= 2
    else danger_order[:]:
        if not judge_save:

```

```

        eliminated = bottom2[0] # 最危险者淘汰 (score 最大)

    else:
        bdf = wdf[wdf["contestant"].isin(bottom2)].copy()
        Js = bdf.set_index("contestant")["J"].to_dict()
        a, b = bottom2[0], bottom2[1] if len(bottom2) > 1
    else (bottom2[0], bottom2[0])
        wa = np.exp(-kappa * float(Js[a]))
        wb = np.exp(-kappa * float(Js[b]))
        pa = wa / (wa + wb) if (wa + wb) > 0 else 0.5
        eliminated = a if rng.random() < pa else b

    else:
        raise ValueError("rule 只能是 percent 或 rank")

    # 记录日志 (用于后续计算 "淘汰者在评委名次中的分位")
    wdf2 = wdf.copy()
    wdf2["season"] = season_id
    wdf2["week"] = w
    wdf2["rule"] = rule + ("JS" if judge_save else "")
    wdf2["alive_n"] = len(alive)
    logs.append(wdf2)

    # 更新存活集合
    elim_pred[w] = [eliminated]
    if eliminated in alive:
        alive.remove(eliminated)

    # 最后一周做最终排名 (冠军在第一)
    last_w = weeks[-1]
    last_df = votes_season[votes_season["week"] == last_w].copy()
    last_df = last_df[last_df["contestant"].isin(alive)].copy()
    if last_df.empty:
        final_order = []
    else:
        if rule == "percent":
            last_df["score"] = last_df["q_hat"].astype(float) +
last_df["p_hat"].astype(float)
            last_df = last_df.sort_values("score", ascending=False)
        # 分高更好
        else:
            last_df["rJ"] = rank_desc(last_df["J"].astype(float))
            last_df["rV"] =
rank_desc(last_df["p_hat"].astype(float))
            last_df["score"] = last_df["rJ"] + last_df["rV"]

```

```

        last_df = last_df.sort_values("score", ascending=True)
# 名次和小更好
        final_order = last_df["contestant"].tolist()

        log_df = pd.concat(logs, ignore_index=True) if logs else
pd.DataFrame()
        return elim_pred, final_order, log_df

# =====
# 6. 运行全赛季: percent vs rank, 并做指标统计
# =====
def judge_percentile_of_elim(log_df: pd.DataFrame, elim_pred:
dict):
    """
    计算每周淘汰者在当周评委名次中的分位数:
    - rank_J = 1 最好
    - 分位数 phi = (rank_J-1)/(n_alive-1), 越接近 1 表示越接近 “评委
最差”
    """
    rows = []
    if log_df.empty:
        return pd.DataFrame(rows)

    # 按周分组计算评委名次
    for (s, w, rule_name), g in log_df.groupby(["season", "week",
"rule"]):
        g = g.copy()
        g["rankJ"] = (-
g["J"].astype(float)).rank(method="min").astype(int) # 1 最好
        n = len(g)
        if n <= 1:
            continue
        elims = elim_pred.get(int(w), [])
        for e in elims:
            if e in g["contestant"].values:
                rj = int(g.loc[g["contestant"] == e,
"rankJ"].iloc[0])
                phi = (rj - 1) / (n - 1)
                rows.append({"season": int(s), "week": int(w),
"rule": rule_name,
                                "contestant": e, "rankJ": rj,
"alive_n": n, "phi": phi})
        return pd.DataFrame(rows)

```

```

def elimination_disagree_rate(elimA: dict, elimB: dict):
    """计算两套规则预测淘汰者不一致的周占比（只统计两边都给出淘汰的周）"""
    weeks =
sorted(set(elimA.keys()).intersection(set(elimB.keys()))))
    if not weeks:
        return np.nan, 0
    total = 0
    diff = 0
    for w in weeks:
        if len(elimA[w]) == 0 or len(elimB[w]) == 0:
            continue
        total += 1
        if elimA[w][0] != elimB[w][0]:
            diff += 1
    return (diff / total if total > 0 else np.nan), total

def spearman_rank_corr(orderA: list, orderB: list):
    """决赛周名次序列的 Spearman 相关（只比较共同出现的人）"""
    if not orderA or not orderB:
        return np.nan
    common = [x for x in orderA if x in orderB]
    if len(common) <= 2:
        return np.nan
    ra = {nm: i+1 for i, nm in enumerate(orderA)}
    rb = {nm: i+1 for i, nm in enumerate(orderB)}
    xa = np.array([ra[nm] for nm in common], float)
    xb = np.array([rb[nm] for nm in common], float)
    # Spearman: 对名次再做 Pearson
    xa_r = pd.Series(xa).rank().values
    xb_r = pd.Series(xb).rank().values
    if np.std(xa_r) == 0 or np.std(xb_r) == 0:
        return np.nan
    return float(np.corrcoef(xa_r, xb_r)[0, 1])

# 结果容器
season_metrics = []
phi_all = []

# κ 用于 Judge Save 强度；数值越大越接近“必淘汰评委更低分者”
KAPPA = 50.0
rng = np.random.default_rng(123)

```

```

for s in sorted(votes_all["season"].unique()):
    vs = votes_all[votes_all["season"] == s].copy()
    if vs["week"].nunique() < 2:
        continue

    # 真实淘汰 (用 0 分推断)
    g = df[df[season_col] == s].copy()
    valid_ws = season_weeks[int(s)]
    elim_real = infer_eliminations_one_season(g, valid_ws)

    # 两套规则回放 (不带 Judge Save)
    elim_p, final_p, log_p = simulate_season(int(s), vs, elim_real,
rule="percent", judge_save=False, rng=rng)
    elim_r, final_r, log_r = simulate_season(int(s), vs, elim_real,
rule="rank", judge_save=False, rng=rng)

    # Judge Save 版本 (两套规则各自加 JS, 便于比较 "救场" 改动效果)
    elim_p_js, final_p_js, log_p_js = simulate_season(int(s), vs,
elim_real, rule="percent", judge_save=True, kappa=KAPPA, rng=rng)
    elim_r_js, final_r_js, log_r_js = simulate_season(int(s), vs,
elim_real, rule="rank", judge_save=True, kappa=KAPPA, rng=rng)

    # 差异率
    disagree_pr, n_weeks = elimination_disagree_rate(elim_p,
elim_r)
    disagree_pr_js, n_weeks_js =
elimination_disagree_rate(elim_p_js, elim_r_js)

    # 决赛名次相关
    corr_pr = spearman_rank_corr(final_p, final_r)
    corr_pr_js = spearman_rank_corr(final_p_js, final_r_js)

    season_metrics.append({
        "season": int(s),
        "weeks_compared": int(n_weeks),
        "disagree_rate_percent_vs_rank": disagree_pr,
        "weeks_compared_JS": int(n_weeks_js),
        "disagree_rate_percentJS_vs_rankJS": disagree_pr_js,
        "final_spearman_percent_vs_rank": corr_pr,
        "final_spearman_percentJS_vs_rankJS": corr_pr_js,
        "champion_percent": final_p[0] if final_p else "",
        "champion_rank": final_r[0] if final_r else "",
        "champion_percentJS": final_p_js[0] if final_p_js else "",
        "champion_rankJS": final_r_js[0] if final_r_js else "",
    })

```

```

    })

    # 评委分位数  $\phi$  (淘汰者在评委名次中的位置) 用于判断 “更偏观众
    吗”
    phi_p = judge_percentile_of_elim(log_p, elim_p)
    phi_r = judge_percentile_of_elim(log_r, elim_r)
    phi_pjs = judge_percentile_of_elim(log_p_js, elim_p_js)
    phi_rjs = judge_percentile_of_elim(log_r_js, elim_r_js)
    phi_all.append(phi_p)
    phi_all.append(phi_r)
    phi_all.append(phi_pjs)
    phi_all.append(phi_rjs)

season_metrics_df = pd.DataFrame(season_metrics)
phi_df = pd.concat(phi_all, ignore_index=True) if phi_all else
pd.DataFrame()

# 输出指标表
metrics_csv = os.path.join(RESULT_DIR, "Q2_season_metrics.csv")
season_metrics_df.to_csv(metrics_csv, index=False, encoding="utf-8-
sig")

phi_csv = os.path.join(RESULT_DIR,
"Q2_eliminated_judge_percentile_phi.csv")
phi_df.to_csv(phi_csv, index=False, encoding="utf-8-sig")

# =====
# 7. 可视化 (保存, 不展示)
# =====
def save_bar_disagree(season_metrics_df: pd.DataFrame):
    """每赛季: percent vs rank 淘汰分歧率柱状图 (含 JS 版本对比) """
    d = season_metrics_df.sort_values("season").copy()
    if d.empty:
        return

    x = np.arange(len(d))
    width = 0.42

    fig, ax = plt.subplots(figsize=(14, 5), dpi=200)
    ax.bar(x - width/2, d["disagree_rate_percent_vs_rank"],
width=width, label="Percent vs Rank")
    ax.bar(x + width/2, d["disagree_rate_percentJS_vs_rankJS"],
width=width, label=f"Percent+JS vs Rank+JS ( $\kappa=\{KAPPA\}$ )")

```

```

    ax.set_title("各赛季：两套规则的周淘汰分歧率（越高表示规则影响越大）", fontsize=13, pad=12)
    ax.set_xlabel("Season")
    ax.set_ylabel("Disagree Rate")
    ax.set_xticks(x)
    ax.set_xticklabels(d["season"].tolist(), rotation=0,
    fontsize=8)
    ax.set_ylim(0, 1.02)
    ax.legend()

    fig.tight_layout()
    out = os.path.join(RESULT_DIR,
    "Q2_disagree_rate_by_season.png")
    fig.savefig(out, bbox_inches="tight")
    plt.close(fig)

def save_phi_boxplot(phi_df: pd.DataFrame):
    """
    淘汰者的评委分位数 phi 箱线图:
    - phi 越接近 1: 淘汰者越接近评委最差 => 规则越尊重评委
    - phi 越接近 0: 淘汰者越接近评委最好 => 更容易出现“观众翻盘”
    """

    if phi_df.empty:
        return

    # 为了对比清晰，按规则名聚合
    order = ["percent", "rank", "percent+JS", "rank+JS"]
    # 实际 rule 字段长这样: percent 或 rank 或 percent+JS 等
    # 这里做标准化
    tmp = phi_df.copy()
    tmp["rule2"] = tmp["rule"].replace({
        "percent": "percent",
        "rank": "rank",
        "percent+JS": "percent+JS",
        "rank+JS": "rank+JS",
    })
    # 某些情况下 rule 字段可能已经是 percent+JS 之类
    tmp.loc[tmp["rule"].str.contains("percent") &
    tmp["rule"].str.contains("JS"), "rule2"] = "percent+JS"
    tmp.loc[tmp["rule"].str.contains("rank") &
    tmp["rule"].str.contains("JS"), "rule2"] = "rank+JS"
    tmp.loc[tmp["rule"].str.fullmatch("percent"), "rule2"] =
    "percent"

```

```

tmp.loc[tmp["rule"].str.fullmatch("rank"), "rule2"] = "rank"

data = []
labels = []
for r in order:
    vals = tmp.loc[tmp["rule2"] == r, "phi"].dropna().values
    if len(vals) > 0:
        data.append(vals)
        labels.append(r)

if len(data) < 2:
    return

fig, ax = plt.subplots(figsize=(10, 5), dpi=200)
ax.boxplot(data, labels=labels, showfliers=False)
ax.set_title("淘汰者在评委名次中的分位数  $\phi$  分布（比较规则偏向性）", fontsize=13, pad=12)
ax.set_xlabel("Rule")
ax.set_ylabel(" $\phi = (\text{rankJ}-1)/(\text{n}-1)$  （越接近 1 越像淘汰评委低分者）")
ax.set_ylim(-0.05, 1.05)

fig.tight_layout()
out = os.path.join(RESULT_DIR, "Q2_phi_boxplot_rules.png")
fig.savefig(out, bbox_inches="tight")
plt.close(fig)

def save_final_rank_corr(season_metrics_df: pd.DataFrame):
    """决赛名次相关 (Spearman) 按赛季折线图"""
    d = season_metrics_df.sort_values("season").copy()
    if d.empty:
        return

    fig, ax = plt.subplots(figsize=(14, 5), dpi=200)
    ax.plot(d["season"], d["final_spearman_percent_vs_rank"],
            marker="o", linewidth=2, label="Percent vs Rank")
    ax.plot(d["season"], d["final_spearman_percentJS_vs_rankJS"],
            marker="o", linewidth=2,
            label=f"Percent+JS vs Rank+JS ( $\kappa = \{\text{KAPPA}\}$ )")

    ax.set_title("各赛季：两套规则下决赛名次序列 Spearman 相关（越低差异越大）", fontsize=13, pad=12)
    ax.set_xlabel("Season")
    ax.set_ylabel("Spearman correlation")

```

```

ax.set_ylim(-1.05, 1.05)
ax.legend()

fig.tight_layout()
out = os.path.join(RESULT_DIR,
"Q2_final_rank_spearman_by_season.png")
fig.savefig(out, bbox_inches="tight")
plt.close(fig)

def save_champion_compare_table(season_metrics_df: pd.DataFrame):
    """输出一个冠军对照表 (CSV) """
    d = season_metrics_df[[
        "season", "champion_percent", "champion_rank",
"champion_percentJS", "champion_rankJS"
    ]].copy()
    out = os.path.join(RESULT_DIR, "Q2_champion_comparison.csv")
    d.to_csv(out, index=False, encoding="utf-8-sig")

def save_focus_season_elim_trace(season_id: int, season_metrics_df:
pd.DataFrame, votes_all: pd.DataFrame):
    """
    为特定赛季画“淘汰路径分歧示意图”：
    - x 轴周次
    - y 轴：两套规则预测淘汰者的评委分位数  $\phi$ （或用点标记）
    目的：直观看到规则在关键周的差异
    """
    # 重新做回放（保证数据完整），并输出 phi 序列
    vs = votes_all[votes_all["season"] == season_id].copy()
    if vs.empty or vs["week"].nunique() < 2:
        return

    g = df[df[season_col] == season_id].copy()
    valid_ws = season_weeks[int(season_id)]
    elim_real = infer_eliminations_one_season(g, valid_ws)

    rng_local = np.random.default_rng(2026)

    elim_p, _, log_p = simulate_season(season_id, vs, elim_real,
rule="percent", judge_save=False, rng=rng_local)
    elim_r, _, log_r = simulate_season(season_id, vs, elim_real,
rule="rank", judge_save=False, rng=rng_local)

    phi_p = judge_percentile_of_elim(log_p, elim_p)
    phi_r = judge_percentile_of_elim(log_r, elim_r)

```

```

if phi_p.empty or phi_r.empty:
    return

# 对齐周次
p_series = phi_p.groupby("week")["phi"].mean()
r_series = phi_r.groupby("week")["phi"].mean()
weeks = sorted(set(p_series.index).union(set(r_series.index)))

yp = [p_series.get(w, np.nan) for w in weeks]
yr = [r_series.get(w, np.nan) for w in weeks]

fig, ax = plt.subplots(figsize=(12, 5), dpi=200)
ax.plot(weeks, yp, marker="o", linewidth=2, label="Percent")
ax.plot(weeks, yr, marker="o", linewidth=2, label="Rank")

ax.set_title(f"Season {season_id}: 两套规则下“淘汰者评委分位数
 $\phi$ ”周序列（ $\phi$  越高越淘汰评委低分者）", fontsize=12, pad=12)
ax.set_xlabel("Week")
ax.set_ylabel(" $\phi$ ")
ax.set_ylim(-0.05, 1.05)
ax.legend()

fig.tight_layout()
out = os.path.join(RESULT_DIR,
f"Q2_focus_season_{season_id}_phi_trace.png")
fig.savefig(out, bbox_inches="tight")
plt.close(fig)

# 生成图
save_bar_disagree(season_metrics_df)
save_phi_boxplot(phi_df)
save_final_rank_corr(season_metrics_df)
save_champion_compare_table(season_metrics_df)

# 重点赛季（题面常提 2、27；若不存在就跳过）
for focus in [2, 4, 11, 27, 28, 34]:
    if focus in season_metrics_df["season"].values:
        save_focus_season_elim_trace(focus, season_metrics_df,
votes_all)

# =====
# 8. 最终提示
# =====

```

```

print("运行完成 )

print(f"- 指标表: {metrics_csv}")
print(f"- 淘汰者评委分位数  $\Phi$ : {phi_csv}")
print(f"- 图片均已保存到: {RESULT_DIR}")
print("主要图片: ")
print(" - Q2_disagree_rate_by_season.png")
print(" - Q2_phi_boxplot_rules.png")
print(" - Q2_final_rank_spearman_by_season.png")
print(" - Q2_focus_season*_phi_trace.png (若对应赛季存在)")

```

问题 3

问题 3 分析

1) 思路框架：将“评委评价”和“粉丝投票”视为两套行为系统，分离并比较专业舞者与名人特征的作用路径

第 3 问的核心不再是“规则是否公平”，而是进一步追问：在既定的赛制和规则下，**谁在影响评委评分，谁又在影响粉丝投票**，以及这两套影响机制是否一致。赛题明确给出一个重要背景：评委评分主要体现舞蹈技术与专业表现，而粉丝投票更容易受到名人知名度、形象和背景等非技术因素的影响。这一设定意味着，如果仍然把“比赛表现”当作一个单一结果变量来分析，就会掩盖评委与观众行为逻辑的本质差异。因此，第 3 问的首要思想是**将评委评分与粉丝投票在模型层面彻底拆分**，分别作为两个因变量进行建模，再对比同一组解释变量在两类模型中的作用方向和强度。

在前两问中，我们已经完成了两项关键铺垫：一是通过第 1 问反推出了每位选手每周的粉丝投票估计值，从而首次获得了可用于量化分析的“观众偏好”指标；二是在第 2 问中确认，评委评分与粉丝投票在多赛季、多案例中确实可能产生系统性分歧。这使得第 3 问可以在数据层面直接回答一个更深层的问题：**这种分歧究竟来自哪里**。从题面与数据字段看，影响因素主要分为两类：一类是名人自身的固有特征（如年龄、职业领域、性别、名气类型等），另一类是其搭档的专业舞者。后者尤其关键，因为同一名专业舞者可能在不同赛季、不同名人身上反复出现，这为识别“舞者是否具有系统性带人能力”提供了天然的准实验结构。

因此，第 3 问的总体思路是：在“周—选手”为观测单位的纵向数据框架下，分别构建**评委评分模型**与**粉丝投票模型**，将名人特征作为固定效应，将专业舞者与赛季作为随机效应或分层结构引入模型中，通过比较两类模型的系数结构与显著性差异，回答以下三个紧密相关的问题：专业舞者是否显著提升技术评分或人气；名人特征对评委与观众的影响是否一致；评委评分本身能否有效转化为粉丝投票支持。

解题思路：

2) 数学模型构建：分离“技术评价方程”与“观众偏好方程”的双系统建模框架

2.1 变量定义与数据结构（对齐赛题附件字段）

以赛季 s 、周次 t 、参赛选手 i 为索引单位。每个观测对应一条“该选手在该周仍在比赛中的记录”。评委总评分记为 $J_{i,t}$ ，该变量已在前两问中由 `weekX_judgeY_score` 字段

求和得到。粉丝投票使用第 1 问输出的投票占比估计值 $\hat{p}_{i,t}$ ，其满足 $0 \leq \hat{p}_{i,t} \leq 1$ 且

$$\sum_i \hat{p}_{i,t} = 1。$$

名人特征向量记为 $\mathbf{Z}_i$ ，可包含年龄（Age）、性别（Gender）、职业类别（Industry，以虚拟变量表示）等在数据中可直接提取的信息。专业舞者记为 $d(i)$ ，即选手 i 在该赛季对应的 ballroom partner；赛季编号记为 $s(i,t)$ ，用于刻画不同赛季在评分尺度、观众结构上的系统差异。

2.2 技术评价模型：解释评委评分的分层线性模型

评委评分被视为对舞蹈技术与专业表现的直接评价，因此其模型的目标是识别“哪些因素能稳定提升技术得分”。考虑到同一专业舞者可能反复出现、同一赛季内评分环境相对一致，采用分层模型能够避免把舞者差异误判为名人特征效应。评委评分模型可写为：

$$J_{i,t} = \alpha_0 + \boldsymbol{\alpha}^\top \mathbf{Z}_i + u_{d(i)} + v_{s(i,t)} + \varepsilon_{i,t} \quad (29)$$

其中， $\boldsymbol{\alpha}$ 表示名人特征对评委评分的固定效应， $u_{d(i)}$ 为专业舞者的随机效应，用于刻画不同舞者在“提升技术评分”方面的系统差异， $v_{s(i,t)}$ 为赛季层面的随机效应， $\varepsilon_{i,t}$ 为个体误差项。若 u_d 的方差显著大于 0，则说明专业舞者对评委评分具有不可忽略的影响。

2.3 观众偏好模型：解释粉丝投票的非技术驱动机制

粉丝投票反映的是观众偏好，其分布通常高度偏态，且为比例型变量。为了保证建模稳定性与可解释性，对投票占比进行对数变换或 logit 变换。以对数变换为例，观众偏好模型设为

$$\log(\hat{p}_{i,t}) = \beta_0 + \beta_1 J_{i,t} + \boldsymbol{\beta}^\top \mathbf{Z}_i + u'_d(i) + v'_{s(i,t)} + \eta_{i,t} \quad (30)$$

该模型中引入 $J_{i,t}$ 的目的，是直接检验“评委评分是否能够转化为粉丝投票”，即技术认可的传播效率。 $\boldsymbol{\beta}$ 则刻画名人特征对观众偏好的直接影响，而 $u'_d(i)$ 表示专业舞者是否能够通过舞台表现、编舞风格或人气加成影响粉丝投票。比较 u_d 与 u'_d 的分布，可以判断舞者的作用是偏“技术型”、偏“人气型”，还是两者兼具。

2.4 作用机制对比的量化指标

为了在论文中清晰呈现差异，引入两个对比指标。第一个是舞者效应比率：

$$R_d = \frac{\text{Var}(u'_d)}{\text{Var}(u_d)} \quad (31)$$

若 $R_d > 1$ ，说明舞者对粉丝投票的影响强于对评委评分的影响；反之则更偏技术。第二个是**技术转化系数** β_1 ，其显著性与大小直接反映“高技术是否一定带来高人气”，为后续制度设计问题提供行为层证据。

3) 模型算法求解：分层估计、效应对照与行为解释的落地流程

3.1 数据整理与样本构建

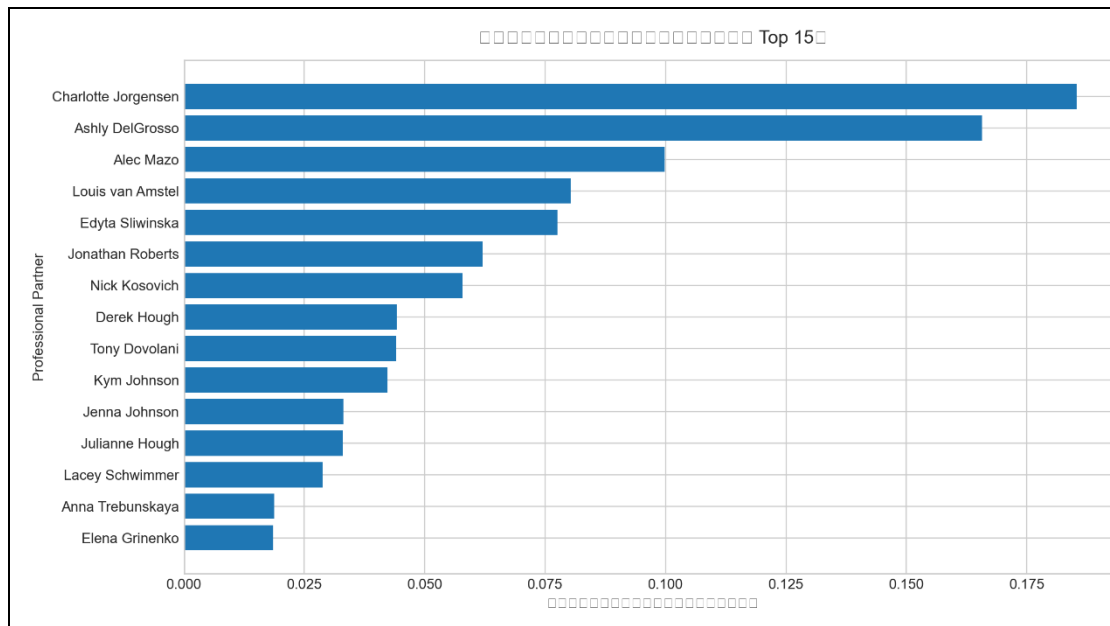
首先仅保留“该周仍在比赛中”的观测，即 $J_{i,t} > 0$ 的记录，避免淘汰后 0 分对模型的结构性干扰。对职业等分类变量进行哑变量编码，并对年龄等连续变量进行标准化处理，以保证系数尺度可比。粉丝投票占比 $\hat{p}_{i,t}$ 过小的观测可加微小正数以避免对数变换中的数值问题。

3.2 分层模型的估计策略

评委评分模型与粉丝投票模型分别采用最大似然或限制最大似然方法进行估计。由于样本量按“选手×周”累积，且随机效应结构简单（舞者与赛季两层），模型在计算上是可行且稳定的。估计完成后，提取舞者随机效应的后验均值 $\hat{u}_d$ 与 $\hat{u}'_d$ ，并进行排序与对照分析。

3.3 结果解释与赛题问题的直接回应

通过比较两类模型中名人特征系数的显著性，可以直接回答“名人背景是否更多影响观众而非评委”；通过比较 u_d 与 u'_d 的分布，可以判断“专业舞者是否在技术或人气上存在系统性差异”；而 β_1 的估计结果则为“评委评分是否能有效引导观众投票”这一隐含问题提供量化证据。这种解释路径与前两问关于规则公平性和争议结果的讨论形成闭环：如果技术难以转化为人气，那么即使制度形式公平，也更容易产生类似第 2 季或第 27 季的分歧结果。



Python 代码:

```
# -*- coding: utf-8 -*-
"""
```

问题 3: 分析 “专业舞者 (ballroom partner)” 与 “名人特征” 对
(1) 评委评分 (技术评价) 和 (2) 粉丝投票 (观众偏好) 的影响差异

实现目标 (面向快速解决实际问题):

1) 读取真实数据: 2026_MCM_Problem_C_Data.csv (根目录, 直接用文件名)

2) 优先读取问题 1 的投票估计结果:

result1/Q1_estimated_votes_long.csv

- 若不存在, 则自动快速估计投票占比 p_hat (用百分比淘汰约束近似, 能跑通)

3) 构建 “周-选手” 长表数据

4) 两个模型:

- 模型 A (评委评分): $J \sim \text{名人特征} + \text{week} + (1/\text{partner}) + \text{season}$ 随机项

- 模型 B (粉丝投票): $\log(p_hat) \sim J + \text{名人特征} + \text{week} + (1/\text{partner}) + \text{season}$ 随机项

5) 输出图表到 result1 文件夹 (先创建), 不展示

依赖:

- pandas, numpy, matplotlib

- cvxpy (仅在未找到 Q1 估计文件时用于自动估票)

- statsmodels (用于 MixedLM 混合效应模型): `pip install statsmodels`

```
import os
import re
```

```

import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import statsmodels.formula.api as smf

# 若需要自动估计投票（没有 Q1 文件）会用到
import cvxpy as cp

# =====
# 0. 输出目录与绘图风格
# =====
RESULT_DIR = "result3"
os.makedirs(RESULT_DIR, exist_ok=True)

plt.style.use("seaborn-v0_8-whitegrid")
np.random.seed(42)

# =====
# 1. 读取真实数据
# =====
DATA_FILE = "2026_MCM_Problem_C_Data.csv"
df = pd.read_csv(DATA_FILE)
cols = df.columns.tolist()

def find_col(candidates, columns):
    """在列名里尽量稳健地找目标列（含大小写兼容）"""
    for c in candidates:
        if c in columns:
            return c
    cols_lower = {x.lower(): x for x in columns}
    for c in candidates:
        if c.lower() in cols_lower:
            return cols_lower[c.lower()]
    raise ValueError(f"找不到列：候选={candidates}")

# 赛季列
season_col = find_col(["season", "Season", "SEASON"], cols)

# 选手列（名人）
try:
    contestant_col = find_col(["celebrity", "Celebrity",

```

```

"contestant", "Contestant", "star", "Star", "name", "Name"], cols)
except Exception:
    # 退而求其次：唯一值多的文本列
    obj_cols = [c for c in cols if df[c].dtype == "object"]
    if not obj_cols:
        raise ValueError("数据中没有明显文本列可作为选手标识。请检查 CSV。")
    contestant_col = sorted(obj_cols, key=lambda c:
df[c].nunique(), reverse=True)[0]
    warnings.warn(f"未识别到标准选手列名，暂用 {contestant_col} 作为选手标识列")

# 专业舞者列 (ballroom partner) ——不同数据版本可能叫 partner /
pro / dancer 等
partner_candidates = [
    "ballroom_partner", "BallroomPartner", "ballroom partner",
    "Ballroom Partner",
    "partner", "Partner", "pro_partner", "ProPartner",
    "professional_partner",
    "pro", "Pro", "dancer", "Dancer", "professional_dancer"
]
partner_col = None
for c in partner_candidates:
    if c in cols:
        partner_col = c
        break
if partner_col is None:
    # 模糊查找：列名里包含 partner 或 pro 或 dancer
    for c in cols:
        cl = c.lower()
        if ("partner" in cl) or ("pro" in cl and "week" not in cl)
or ("dancer" in cl):
            partner_col = c
            break
if partner_col is None:
    raise ValueError("找不到专业舞者 (partner/pro/dancer) 字段，请检查 CSV 列名。")

# 年龄列
age_col = None
for c in ["age", "Age", "AGE"]:
    if c in cols:
        age_col = c
        break

```

```

# 性别列（若没有就忽略）
gender_col = None
for c in ["gender", "Gender", "sex", "Sex"]:
    if c in cols:
        gender_col = c
        break

# 职业/行业列（用于解释“名人背景”）
industry_col = None
industry_candidates = ["industry", "Industry", "occupation",
"Occupation", "profession", "Profession", "job", "Job"]
for c in industry_candidates:
    if c in cols:
        industry_col = c
        break
if industry_col is None:
    # 模糊：包含 occup / profess / indust
    for c in cols:
        cl = c.lower()
        if ("occup" in cl) or ("profess" in cl) or ("industry" in
cl):
            industry_col = c
            break

# 解析 weekX_judgeY_score 列
week_judge_pattern = re.compile(r"week(\d+)_judge(\d+)_score",
re.IGNORECASE)
week_cols = {}
for c in cols:
    m = week_judge_pattern.match(c)
    if m:
        w = int(m.group(1))
        week_cols.setdefault(w, []).append(c)
if not week_cols:
    raise ValueError("未找到 weekX_judgeY_score 列，请检查 CSV 格
式。")

all_weeks = sorted(week_cols.keys())

# 计算每周评委总分  $J_{week\{w\}}$ 
for w in all_weeks:
    df[f"J_week{w}"] = df[week_cols[w]].apply(pd.to_numeric,
errors="coerce").sum(axis=1, skipna=True)

```

```

# 每赛季有效周: 该周至少有一人  $J > 0$ 
season_weeks = {}
for s, g in df.groupby(season_col):
    valid_ws = []
    for w in all_weeks:
        if (g[f"J_week{w}"] > 0).any():
            valid_ws.append(w)
    season_weeks[int(s)] = valid_ws

# =====
# 2. 读入 (或自动生成) Q1 投票估计:  $p_{hat}$ 
# =====
Q1_FILE = os.path.join(RESULT_DIR, "Q1_estimated_votes_long.csv")

def infer_eliminations_one_season(g_season: pd.DataFrame,
valid_weeks):
    """
    用 0 分机制推断淘汰者:
    本周  $J > 0$  且 下周  $J == 0 \Rightarrow$  本周淘汰
    返回 week  $\rightarrow$  [淘汰者名单]
    """
    elim_map = {}
    names = g_season[contestant_col].astype(str).values
    for idx in range(len(valid_weeks) - 1):
        w = valid_weeks[idx]
        w_next = valid_weeks[idx + 1]
        alive_w = g_season[f"J_week{w}"].values > 0
        alive_next = g_season[f"J_week{w_next}"].values > 0
        eliminated = names[(alive_w) & (~alive_next)]
        elim_map[w] = list(eliminated)
    elim_map[valid_weeks[-1]] = []
    return elim_map

def solve_votes_one_season_percent_approx(
    g_season: pd.DataFrame,
    valid_weeks,
    elim_map,
    lam_entropy=0.60,
    lam_smooth=0.40,
    smooth_weight=5.0,
    eps_margin=1e-6,
):

```

```

"""
快速估计投票占比  $p_{\hat{}}$  (用于 Q3 建模) :
约束:  $T=q+p$ , 真实淘汰者  $T$  最小 (允许多淘汰)
目标: 最大熵 + 跨周平滑
返回: 长表  $season, week, contestant, J, q_{\hat{}}, p_{\hat{}}, T_{\hat{}}$ 
"""

season_id = int(g_season[season_col].iloc[0])

contestants = g_season[contestant_col].astype(str).tolist()
n_total = len(contestants)

week_alive_names = {}
week_J = {}
for w in valid_weeks:
    alive_mask = g_season[f"J_week{w}"].values > 0
    alive_names = [contestants[i] for i in range(n_total) if
alive_mask[i]]
    week_alive_names[w] = alive_names
    week_J[w] = g_season.loc[alive_mask,
f"J_week{w}"].values.astype(float)

p_vars = {}
constraints = []
obj_terms = []

for w in valid_weeks:
    m = len(week_alive_names[w])
    p = cp.Variable(m, nonneg=True)
    p_vars[w] = p
    constraints.append(cp.sum(p) == 1)
    # 最大熵: 最小化  $\sum p \log p = -\sum \text{entr}(p)$ 
    obj_terms.append(lam_entropy * cp.sum(-cp.entr(p)))

# 淘汰一致性约束:  $T=q+p$ , 淘汰者  $T \leq$  存活者  $T$ 
for w in valid_weeks:
    elims = elim_map.get(w, [])
    if not elims:
        continue
    alive_names = week_alive_names[w]
    m = len(alive_names)
    if m <= 1:
        continue

J = week_J[w]

```

```

J_sum = float(np.sum(J)) if np.sum(J) > 0 else 1.0
q = J / J_sum

p = p_vars[w]
T = q + p

elim_idx = [alive_names.index(nm) for nm in elims if nm in
alive_names]
if not elim_idx:
    continue
surv_idx = [i for i in range(m) if i not in elim_idx]
if not surv_idx:
    continue

for ei in elim_idx:
    constraints.append(T[ei] <= T[surv_idx] - eps_margin)

# 跨周平滑: 相邻两周都在赛的选手 p 尽量接近
for i in range(1, len(valid_weeks)):
    w_prev = valid_weeks[i - 1]
    w_curr = valid_weeks[i]
    prev_names = week_alive_names[w_prev]
    curr_names = week_alive_names[w_curr]
    common = sorted(set(prev_names).intersection(curr_names))
    if not common:
        continue

    diffs = []
    for nm in common:
        ip = prev_names.index(nm)
        ic = curr_names.index(nm)
        diffs.append(p_vars[w_curr][ic] - p_vars[w_prev][ip])

    diffs = cp.hstack(diffs)
    obj_terms.append(lam_smooth * smooth_weight *
cp.sum_squares(diffs))

prob = cp.Problem(cp.Minimize(cp.sum(obj_terms)), constraints)

solved = False
last_err = None
for solver in ["ECOS", "SCS", "OSQP"]:
    try:
        prob.solve(solver=solver, verbose=False)

```

```

        if prob.status in ["optimal", "optimal_inaccurate"]:
            solved = True
            break
    except Exception as e:
        last_err = e

    if not solved:
        raise RuntimeError(f"赛季{season_id} 投票估计失败: {last_err}")

    rows = []
    for w in valid_weeks:
        alive_names = week_alive_names[w]
        J = week_J[w]
        J_sum = float(np.sum(J)) if np.sum(J) > 0 else 1.0
        q = J / J_sum
        p_hat = np.maximum(p_vars[w].value, 0)
        p_hat = p_hat / (p_hat.sum() if p_hat.sum() > 0 else 1.0)
        T_hat = q + p_hat

        for k, nm in enumerate(alive_names):
            rows.append({
                "season": season_id,
                "week": int(w),
                "contestant": nm,
                "J": float(J[k]),
                "q_hat": float(q[k]),
                "p_hat": float(p_hat[k]),
                "T_hat": float(T_hat[k]),
            })

    return pd.DataFrame(rows)

# 读取或生成 votes_all (含 p_hat)
if os.path.exists(Q1_FILE):
    votes_all = pd.read_csv(Q1_FILE)
    # 统一列名
    votes_all.columns = [c.strip() for c in votes_all.columns]
    # 兼容不同大小写
    col_map = {}
    for c in votes_all.columns:
        if c.lower() == "season": col_map[c] = "season"
        if c.lower() == "week": col_map[c] = "week"
        if c.lower() == "contestant": col_map[c] = "contestant"

```

```

        if c.lower() == "p_hat": col_map[c] = "p_hat"
        if c.lower() == "q_hat": col_map[c] = "q_hat"
        if c.lower() == "t_hat": col_map[c] = "T_hat"
    votes_all = votes_all.rename(columns=col_map)
    print(f"已读取问题 1 投票估计: {Q1_FILE}")
else:
    print("未找到问题 1 投票估计文件, 将自动估计 p_hat (可能稍慢, 但可运行)。")
    all_votes = []
    for s in sorted(season_weeks.keys()):
        g = df[df[season_col] == s].copy()
        valid_ws = season_weeks[int(s)]
        if len(valid_ws) < 2:
            continue
        elim_map = infer_eliminations_one_season(g, valid_ws)
        votes_long = solve_votes_one_season_percent_approx(g,
valid_ws, elim_map)
        all_votes.append(votes_long)
    votes_all = pd.concat(all_votes, ignore_index=True)
    votes_all.to_csv(Q1_FILE, index=False, encoding="utf-8-sig")
    print(f"已输出问题 1 投票估计: {Q1_FILE}")

votes_all["season"] = votes_all["season"].astype(int)
votes_all["week"] = votes_all["week"].astype(int)
votes_all["contestant"] = votes_all["contestant"].astype(str)

# =====
# 3. 构建问题 3 所需的“周-选手”长表: 合并名人特征与舞者信息
# =====
# 从原始 df 中提取“选手(名人)-赛季”的静态信息
# 注意: 原始 df 每行是一个“选手-赛季”, 包含年龄/行业/舞者等
static_cols = [season_col, contestant_col, partner_col]
if age_col: static_cols.append(age_col)
if gender_col: static_cols.append(gender_col)
if industry_col: static_cols.append(industry_col)

static_df = df[static_cols].copy()
static_df = static_df.rename(columns={
    season_col: "season",
    contestant_col: "contestant",
    partner_col: "partner",
})
static_df["contestant"] = static_df["contestant"].astype(str)

```

```

static_df["partner"] = static_df["partner"].astype(str)

if age_col:
    static_df = static_df.rename(columns={age_col: "age"})
    static_df["age"] = pd.to_numeric(static_df["age"],
errors="coerce")

if gender_col:
    static_df = static_df.rename(columns={gender_col: "gender"})
    static_df["gender"] = static_df["gender"].astype(str)

if industry_col:
    static_df = static_df.rename(columns={industry_col:
"industry"})
    static_df["industry"] = static_df["industry"].astype(str)

# 合并 votes_all (周-选手) 与 static_df (选手-赛季)
long_df = votes_all.merge(static_df, on=["season", "contestant"],
how="left")

# 只保留在赛周 (J>0) ——这是第 3 问核心分析的有效样本
long_df = long_df[pd.to_numeric(long_df["J"],
errors="coerce").fillna(0) > 0].copy()

# 构造 log(p_hat) 作为观众偏好变量 (避免 log(0))
EPS = 1e-12
long_df["log_p"] = np.log(np.clip(long_df["p_hat"].astype(float),
EPS, 1.0))

# 清理 partner 缺失
long_df["partner"] =
long_df["partner"].fillna("Unknown").astype(str)
long_df["season"] = long_df["season"].astype(int)
long_df["week"] = long_df["week"].astype(int)

# 对年龄缺失做简单处理: 用赛季内中位数填充 (比全局均值更稳)
if "age" in long_df.columns:
    long_df["age"] =
long_df.groupby("season")["age"].transform(lambda x:
x.fillna(x.median()))
    # 若某赛季全缺失, 再用全局中位数
    long_df["age"] = long_df["age"].fillna(long_df["age"].median())

# 对 industry/gender 缺失

```

```

if "industry" in long_df.columns:
    long_df["industry"] = long_df["industry"].replace({"nan":
np.nan}).fillna("Unknown").astype(str)
if "gender" in long_df.columns:
    long_df["gender"] = long_df["gender"].replace({"nan":
np.nan}).fillna("Unknown").astype(str)

# 输出用于复核的数据
long_out = os.path.join(RESULT_DIR, "Q3_modeling_long_data.csv")
long_df.to_csv(long_out, index=False, encoding="utf-8-sig")

# =====
# 4. 混合效应模型：评委评分 vs 粉丝投票
# - random intercept: partner
# - season 作为方差分量 vc（控制赛季整体差异）
# =====
# 构造公式：根据可用特征动态拼接
fixed_terms = ["week"]
if "age" in long_df.columns:
    fixed_terms.append("age")
if "gender" in long_df.columns:
    fixed_terms.append("C(gender)")
if "industry" in long_df.columns:
    # 行业类别通常较多，为避免过度复杂，可选只保留 TopK，其余归为
    Other
    # 这里做一个非常实用的简化：保留出现次数最多的 15 类
    top_k = 15
    top_ind =
long_df["industry"].value_counts().head(top_k).index.tolist()
    long_df["industry2"] =
np.where(long_df["industry"].isin(top_ind), long_df["industry"],
"Other")
    fixed_terms.append("C(industry2)")
else:
    long_df["industry2"] = "Unknown"

# 评委评分模型（技术评价系统）
judge_formula = "J ~ " + " + ".join(fixed_terms)

# 粉丝投票模型（观众偏好系统），加入评委评分 J 来检验“技术转化为人
气”的程度
fan_formula = "log_p ~ J + " + " + ".join(fixed_terms)

```

```

# 方差分量: season (随机截距)
vc = {"season": "0 + C(season)"}

# 拟合模型 A: 评委评分
# groups=partner: 表示“每个专业舞者一个随机截距”
print("\n 开始拟合模型 A (评委评分 MixedLM) ...")
model_j = smf.mixedlm(judge_formula, long_df,
groups=long_df["partner"], vc_formula=vc, re_formula="1")
res_j = model_j.fit(method="lbfgs", maxiter=200, disp=False)
print("模型 A 拟合完成。")

# 拟合模型 B: 粉丝投票
print("\n 开始拟合模型 B (粉丝投票 log(p_hat) MixedLM) ...")
model_f = smf.mixedlm(fan_formula, long_df,
groups=long_df["partner"], vc_formula=vc, re_formula="1")
res_f = model_f.fit(method="lbfgs", maxiter=200, disp=False)
print("模型 B 拟合完成。")

# 保存模型摘要 (文本)
with open(os.path.join(RESULT_DIR,
"Q3_modelA_judgescore_summary.txt"), "w", encoding="utf-8") as f:
    f.write(str(res_j.summary()))
with open(os.path.join(RESULT_DIR,
"Q3_modelB_fanscore_summary.txt"), "w", encoding="utf-8") as f:
    f.write(str(res_f.summary()))

# 提取固定效应系数表
def coef_table(res):
    """把固定效应系数/标准误/置信区间整理成表格"""
    params = res.params
    bse = res.bse
    out = pd.DataFrame({
        "term": params.index,
        "coef": params.values,
        "se": bse.values
    })
    out["ci_low"] = out["coef"] - 1.96 * out["se"]
    out["ci_high"] = out["coef"] + 1.96 * out["se"]
    return out

coef_j = coef_table(res_j)
coef_f = coef_table(res_f)

coef_j.to_csv(os.path.join(RESULT_DIR,

```

```

"Q3_modelA_fixed_effects.csv"), index=False, encoding="utf-8-sig")
coef_f.to_csv(os.path.join(RESULT_DIR,
"Q3_modelB_fixed_effects.csv"), index=False, encoding="utf-8-sig")

# =====
# 5. 提取专业舞者随机效应 (partner random intercept)
# =====
def extract_partner_random_intercepts(res):
    """
    statsmodels MixedLM 的随机效应结果在 res.random_effects 中
    每个 group (partner) 对应一个字典, 通常包含 'Group' 或类似键
    这里做健壮提取: 取第一个值作为随机截距
    """
    re = res.random_effects
    rows = []
    for partner, d in re.items():
        if isinstance(d, (pd.Series, dict)):
            # 取第一个元素当随机截距
            val = float(list(d.values())[0]) if isinstance(d, dict)
        else float(d.iloc[0])
        else:
            val = float(d)
        rows.append({"partner": str(partner), "rand_intercept":
val})
    out = pd.DataFrame(rows)
    out = out.sort_values("rand_intercept", ascending=False)
    return out

partner_j = extract_partner_random_intercepts(res_j)
partner_f = extract_partner_random_intercepts(res_f)

partner_j.to_csv(os.path.join(RESULT_DIR,
"Q3_partner_effect_on_judgescore.csv"), index=False,
encoding="utf-8-sig")
partner_f.to_csv(os.path.join(RESULT_DIR,
"Q3_partner_effect_on_fanvote.csv"), index=False, encoding="utf-8-
sig")

# =====
# 6. 可视化 (保存, 不展示)
# =====
def save_bar_top_partners(partner_df: pd.DataFrame, title: str,

```

```

outfile: str, top_n: int = 15):
    """绘制专业舞者随机截距 TopN 柱状图"""
    d = partner_df.head(top_n).copy()
    d = d.sort_values("rand_intercept", ascending=True) # 横向更美
    观
    fig, ax = plt.subplots(figsize=(11, 6), dpi=200)
    ax.barh(d["partner"], d["rand_intercept"])
    ax.set_title(title, fontsize=13, pad=12)
    ax.set_xlabel("随机截距（越大表示带来的系统性提升越强）")
    ax.set_ylabel("Professional Partner")
    fig.tight_layout()
    fig.savefig(os.path.join(RESULT_DIR, outfile),
bbox_inches="tight")
    plt.close(fig)

def save_bar_bottom_partners(partner_df: pd.DataFrame, title: str,
outfile: str, bottom_n: int = 15):
    """绘制专业舞者随机截距 BottomN 柱状图"""
    d = partner_df.tail(bottom_n).copy()
    d = d.sort_values("rand_intercept", ascending=True)
    fig, ax = plt.subplots(figsize=(11, 6), dpi=200)
    ax.barh(d["partner"], d["rand_intercept"])
    ax.set_title(title, fontsize=13, pad=12)
    ax.set_xlabel("随机截距（越小表示带来的系统性提升越弱/甚至拖
累）")
    ax.set_ylabel("Professional Partner")
    fig.tight_layout()
    fig.savefig(os.path.join(RESULT_DIR, outfile),
bbox_inches="tight")
    plt.close(fig)

def save_coef_forest(coefA: pd.DataFrame, coefB: pd.DataFrame,
outfile: str):
    """
    对比两个模型的固定效应（森林图）：
    - 只画共同项，并排展示
    - 为避免类别变量太多导致图过长，这里只画：Intercept, week, age,
    J（只在粉丝模型里），以及 gender 的若干项
    """
    # 选取想重点展示的项（你可以按需要加）
    focus_terms = []
    for t in ["Intercept", "week", "age", "J"]:
        focus_terms.append(t)
    # 把性别/行业类别过多的问题限制一下

```

```

for t in coefA["term"].tolist():
    if "C(gender)" in t:
        focus_terms.append(t)
# 行业可选展示前 5 个 (如果有)
ind_terms = [t for t in coefA["term"].tolist() if
"C(industry2)" in t]
focus_terms += ind_terms[:5]

A = coefA[coefA["term"].isin(focus_terms)].copy()
B = coefB[coefB["term"].isin(focus_terms)].copy()

# 合并, 方便对齐画图
terms = sorted(set(A["term"]).union(set(B["term"])))
terms = [t for t in terms if t in focus_terms]

y = np.arange(len(terms))
fig, ax = plt.subplots(figsize=(12, 0.5 * len(terms) + 2.5),
dpi=200)

def get_row(C, term):
    r = C[C["term"] == term]
    if r.empty:
        return None
    return r.iloc[0]

# 模型 A 点位 (评委)
xa, la, ha = [], [], []
for term in terms:
    r = get_row(A, term)
    if r is None:
        xa.append(np.nan); la.append(np.nan); ha.append(np.nan)
    else:
        xa.append(r["coef"]); la.append(r["ci_low"]);
ha.append(r["ci_high"])

# 模型 B 点位 (粉丝)
xb, lb, hb = [], [], []
for term in terms:
    r = get_row(B, term)
    if r is None:
        xb.append(np.nan); lb.append(np.nan); hb.append(np.nan)
    else:
        xb.append(r["coef"]); lb.append(r["ci_low"]);
hb.append(r["ci_high"])

```

```

# 画误差线（不指定颜色，以 Matplotlib 默认色自动区分）
ax.errorbar(xa, y + 0.12, xerr=[np.array(xa) - np.array(la),
np.array(ha) - np.array(xa)],
            fmt="o", capsize=3, label="模型 A: 评委评分(J)")
ax.errorbar(xb, y - 0.12, xerr=[np.array(xb) - np.array(lb),
np.array(hb) - np.array(xb)],
            fmt="o", capsize=3, label="模型 B: 粉丝投票(log
p)")

ax.axvline(0, linestyle="--", linewidth=1)
ax.set_yticks(y)
ax.set_yticklabels(terms, fontsize=9)
ax.set_title("固定效应对比（同一因素对评委 vs 观众的影响方向/强
度）", fontsize=13, pad=12)
ax.set_xlabel("系数估计（含 95%置信区间）")
ax.legend()
fig.tight_layout()
fig.savefig(os.path.join(RESULT_DIR, outfile),
bbox_inches="tight")
plt.close(fig)

def save_scatter_J_vs_logp(long_df: pd.DataFrame, outfile: str):
    """评委评分 vs 粉丝投票（log）散点图 + 线性拟合（整体趋势）"""
    d = long_df[["J", "log_p"]].dropna().copy()
    if len(d) < 50:
        return

    # 简单线性拟合:  $\log_p \sim J$ （用于可视化“技术转化为人气”的总体趋
    势）
    x = d["J"].astype(float).values
    y = d["log_p"].astype(float).values
    b1, b0 = np.polyfit(x, y, 1) #  $y = b1*x + b0$ 

    xs = np.linspace(np.min(x), np.max(x), 200)
    ys = b1 * xs + b0

    fig, ax = plt.subplots(figsize=(9, 6), dpi=200)
    ax.scatter(x, y, s=10, alpha=0.35)
    ax.plot(xs, ys, linewidth=2)

    ax.set_title("评委评分与粉丝投票（log 占比）的关系（总体趋势）",
        fontsize=13, pad=12)
    ax.set_xlabel("评委总分 J")

```

```

ax.set_ylabel("log(粉丝投票占比)")

fig.tight_layout()
fig.savefig(os.path.join(RESULT_DIR, outfile),
bbox_inches="tight")
plt.close(fig)

def save_partner_effect_compare(partner_j: pd.DataFrame, partner_f:
pd.DataFrame, outfile: str, top_n: int = 25):
    """
    同一批专业舞者：对比其对评委评分随机效应 vs 对粉丝投票随机效应
    画散点图：x=judge_random, y=fan_random
    """
    merged = partner_j.merge(partner_f, on="partner", how="inner",
suffixes=("_judge", "_fan"))
    if merged.empty:
        return

    # 取综合影响最大的若干个舞者，避免点太密
    merged["abs_sum"] = merged["rand_intercept_judge"].abs() +
merged["rand_intercept_fan"].abs()
    merged = merged.sort_values("abs_sum",
ascending=False).head(top_n)

    x = merged["rand_intercept_judge"].values
    y = merged["rand_intercept_fan"].values

    fig, ax = plt.subplots(figsize=(8, 7), dpi=200)
    ax.scatter(x, y, s=40, alpha=0.8)
    ax.axhline(0, linestyle="--", linewidth=1)
    ax.axvline(0, linestyle="--", linewidth=1)

    # 标注（只标注少量，避免太乱）
    for _, r in merged.iterrows():
        ax.text(r["rand_intercept_judge"], r["rand_intercept_fan"],
r["partner"], fontsize=8, alpha=0.85)

    ax.set_title("专业舞者随机效应对比：技术提升 vs 人气提升",
fontsize=13, pad=12)
    ax.set_xlabel("对评委评分的随机效应（越大越“带分”）")
    ax.set_ylabel("对粉丝投票的随机效应（越大越“带票”）")

    fig.tight_layout()
    fig.savefig(os.path.join(RESULT_DIR, outfile),

```

```

bbox_inches="tight")
plt.close(fig)

# 6.1 专业舞者：对评委评分的影响 Top/Bottom
save_bar_top_partners(
    partner_j,
    title="专业舞者对评委评分的系统性提升（随机截距 Top 15）",
    outfile="Q3_top_partners_judge_effect.png",
    top_n=15
)
save_bar_bottom_partners(
    partner_j,
    title="专业舞者对评委评分的系统性提升（随机截距 Bottom 15）",
    outfile="Q3_bottom_partners_judge_effect.png",
    bottom_n=15
)

# 6.2 专业舞者：对粉丝投票的影响 Top/Bottom
save_bar_top_partners(
    partner_f,
    title="专业舞者对粉丝投票的系统性提升（随机截距 Top 15）",
    outfile="Q3_top_partners_fan_effect.png",
    top_n=15
)
save_bar_bottom_partners(
    partner_f,
    title="专业舞者对粉丝投票的系统性提升（随机截距 Bottom 15）",
    outfile="Q3_bottom_partners_fan_effect.png",
    bottom_n=15
)

# 6.3 固定效应对比（森林图）
save_coef_forest(coef_j, coef_f,
    outfile="Q3_fixed_effects_forest_compare.png")

# 6.4 评委评分 vs 粉丝投票总体关系
save_scatter_J_vs_logp(long_df, outfile="Q3_scatter_J_vs_logp.png")

# 6.5 舞者“带分” vs “带票”的对照散点（帮助回答：舞者作用更偏技术还是更偏人气）
save_partner_effect_compare(partner_j, partner_f,
    outfile="Q3_partner_effect_compare_scatter.png", top_n=25)

```

```

# =====
# 7. 额外输出：关键量（写报告时非常好用）
# =====
# “技术转化为人气”的核心系数：粉丝模型中 J 的系数
beta_J = float(coef_f.loc[coef_f["term"] == "J", "coef"].iloc[0])
if (coef_f["term"] == "J").any() else np.nan

summary_rows = [{
    "n_rows_long": int(len(long_df)),
    "n_seasons": int(long_df["season"].nunique()),
    "n_partners": int(long_df["partner"].nunique()),
    "fan_model_beta_J": beta_J
}]

summary_df = pd.DataFrame(summary_rows)
summary_df.to_csv(os.path.join(RESULT_DIR, "Q3_key_summary.csv"),
index=False, encoding="utf-8-sig")

print("运行完成 ☑")

print(f"- 长表数据（可复核）：{long_out}")
print(f"- 模型 A 摘要：{os.path.join(RESULT_DIR, 'Q3_modelA_judgescore_summary.txt')}")
print(f"- 模型 B 摘要：{os.path.join(RESULT_DIR, 'Q3_modelB_fanscore_summary.txt')}")
print(f"- 图像输出目录：{RESULT_DIR}")
print("主要图片：")
print(" - Q3_top_partners_judge_effect.png / Q3_top_partners_fan_effect.png")
print(" - Q3_fixed_effects_forest_compare.png")
print(" - Q3_partner_effect_compare_scatter.png")
print(" - Q3_scatter_J_vs_logp.png")

```

问题 4

问题 4 分析

1) 思路框架：将“公平性”与“节目吸引力”视为可量化的双目标，用历史赛季反事实回放检验新制度是否真正改进结果

第 4 问并不是简单要求“提出一个新规则”，而是要求在前几问分析的基础上，给出**一个有数据支撑、逻辑自洽、且能明确说明“为什么更好”**的评分制度。结合前 3 问的结论可以看到：评委评分与粉丝投票代表了两种本质不同的评价体系，其分歧并非偶然，而是长期存在；专业舞者和名人特征的影响也表明，观众行为并不总是与技术评价同步。这意味着，任何新的评分制度如果只是简单调整权重，很容易在“完全偏评委”与“完全偏观众”之间来回摆动，而无法同时满足节目制作方关心的两点：一是避免极端争议结果（如

低技术但高人气者夺冠)，二是保持比赛过程的不确定性与观赏性。

因此，第 4 问的核心思路不是寻找一个“绝对公平”的制度，而是**构造一个可调、可解释的评分机制**，并通过对历史赛季的反事实模拟来证明：在合理参数区间内，该机制能够在不显著削弱节目悬念的前提下，系统性地降低技术评价被完全淹没的概率。前两问已经为这一目标提供了关键工具：第 1 问给出了可用于重算结果的粉丝投票估计，第 2 问给出了在不同规则下回放整季淘汰路径的方法，第 3 问则解释了为什么单纯依赖观众投票容易放大名人背景效应。第 4 问正是要把这些成果整合起来，提出一种“在原有制度基础上小幅调整、但效果可被量化验证”的新规则。

基于赛题背景，一个自然且贴合实际的设计思路是：继续保留评委评分与粉丝投票的共同参与，但通过**非线性处理与二阶段决策**限制极端人气的单边主导作用。具体而言，新制度在第一阶段仍采用百分比形式合成评委评分与粉丝投票，以保持观众的参与感；但在第二阶段，不直接淘汰最低者，而是引入一个“危险区”，由评委在其中进行最终裁决。该结构既继承了节目历史中已经出现过的机制（如倒数两名由评委决定），又为参数化分析提供了空间，使“更公平”与“更吸引粉丝”不再是口号，而是可以通过指标权衡的设计目标。

解题思路：

2) 数学模型构建：带非线性观众约束与二阶段裁决的新评分制度

2.1 新制度的总体结构与变量定义

沿用前几问的符号体系。对赛季 s 、周次 t 、仍在比赛中的选手集合 $\mathcal{A}_{s,t}$ ，评委百分比记为 $q_{i,t}$ ，粉丝投票占比记为 $\hat{p}_{i,t}$ ，两者均已在前文中定义并可直接计算。

新制度的核心在于构造一个**改进后的观众投票贡献函数**，记为 $f(\hat{p}_{i,t})$ ，用以替代原始的线性投票占比，从而抑制极端值对合成结果的过度影响。一个与当前问题高度契合、且便于解释的选择是凹函数，例如平方根变换：

$$f(\hat{p}_{i,t}) = \sqrt{\hat{p}_{i,t}} \quad (32)$$

该函数满足 $f'(\hat{p}) > 0$ 且 $f''(\hat{p}) < 0$ ，意味着观众投票越集中，其边际影响越小，从而在不剥夺观众话语权的情况下限制“人气碾压技术”的极端情形。

2.2 第一阶段：加权非线性合成评分

在第一阶段，对每位选手计算改进后的合成得分：

$$S_{i,t}(\alpha) = \alpha q_{i,t} + (1 - \alpha) f(\hat{p}_{i,t}), \quad 0 \leq \alpha \leq 1 \quad (33)$$

其中 α 为评委权重参数。与传统制度相比，这一表达具有两个重要特点：一是当 α 固定时，评委评分与观众投票仍然同时参与，符合节目长期形成的观众认知；二是即便 $(1 - \alpha)$ 较大，由于 $f(\cdot)$ 的凹性，极端投票优势也不会线性放大，从而避免前几问中观察到的“评委最低但仍长期存活”的情况。

根据 $S_{i,t}(\alpha)$ 的大小排序，定义危险区集合 $\mathcal{B}_{s,t}$ ，例如取合成得分最低的两名：

$$\mathcal{B}_{s,t} = \text{Bottom} 2 \left(\left\{ S_{i,t}(\alpha) \right\}_{i \in \mathcal{A}_{s,t}} \right) \quad (34)$$

这一阶段的设计确保：观众投票依然决定谁“进入危险”，从而维持参与感和戏剧张力。

2.3 第二阶段：评委裁决的概率化建模

在危险区内，由评委决定最终淘汰对象。由于真实节目中评委进行现场投票，而数据中并无此类细节，采用“评委更倾向淘汰技术较弱者”的行为假设作为代理是合理且与前 3 问结论一致的。设危险区为 a, b ，则评委淘汰 a 的概率建模为：

$$\mathbb{P}(e_{s,t} = a | \mathcal{B}_{s,t}) = \frac{\exp(-\kappa J_{a,t})}{\exp(-\kappa J_{a,t}) + \exp(-\kappa J_{b,t})} \quad (35)$$

其中 $\kappa \geq 0$ 表示评委裁决的强度参数。当 $\kappa \rightarrow \infty$ 时，评委几乎必然淘汰评委分更低者；当 $\kappa = 0$ 时，裁决近似随机。该参数化设计的意义在于：它将“评委救场”的力度转化为一个可连续调节、可被敏感性分析的变量，而非一个生硬的制度假设。

2.4 新制度下的“公平性”与“吸引力”指标定义

为了比较新制度与旧制度的优劣，引入两个与前几问自然衔接的指标。公平性指标定义为被淘汰选手的评委分位数：

$$\phi_{s,t} = \frac{r_{e_{s,t},t}^J - 1}{|\mathcal{A}_{s,t}| - 1} \quad (36)$$

其中 r^J 为评委名次。 ϕ 越接近 1，说明被淘汰者越接近评委最低分，制度越尊重技术评价。吸引力指标则通过“结果不确定性”刻画，例如最终冠军在进入赛季前几周的合成得分排名波动，或冠军候选集合的规模。这些指标允许在不依赖主观判断的情况下，对“是否仍保有悬念”进行量化描述。

3) 模型算法求解：基于历史赛季的反事实模拟与参数敏感性分析

3.1 历史赛季的反事实回放流程

算法上，沿用第 2 问构建的赛季回放框架。对每一个历史赛季 s ，在保持 $J_{i,t}$ 与 $\hat{p}_{i,t}$ 不变的前提下，用新制度替代原有淘汰规则，逐周递推生成淘汰序列与最终名次。该过程严格遵循真实赛程结构，仅改变淘汰决策逻辑，因此可以被视为“如果当年采用新制度，结果会如何”的反事实实验。

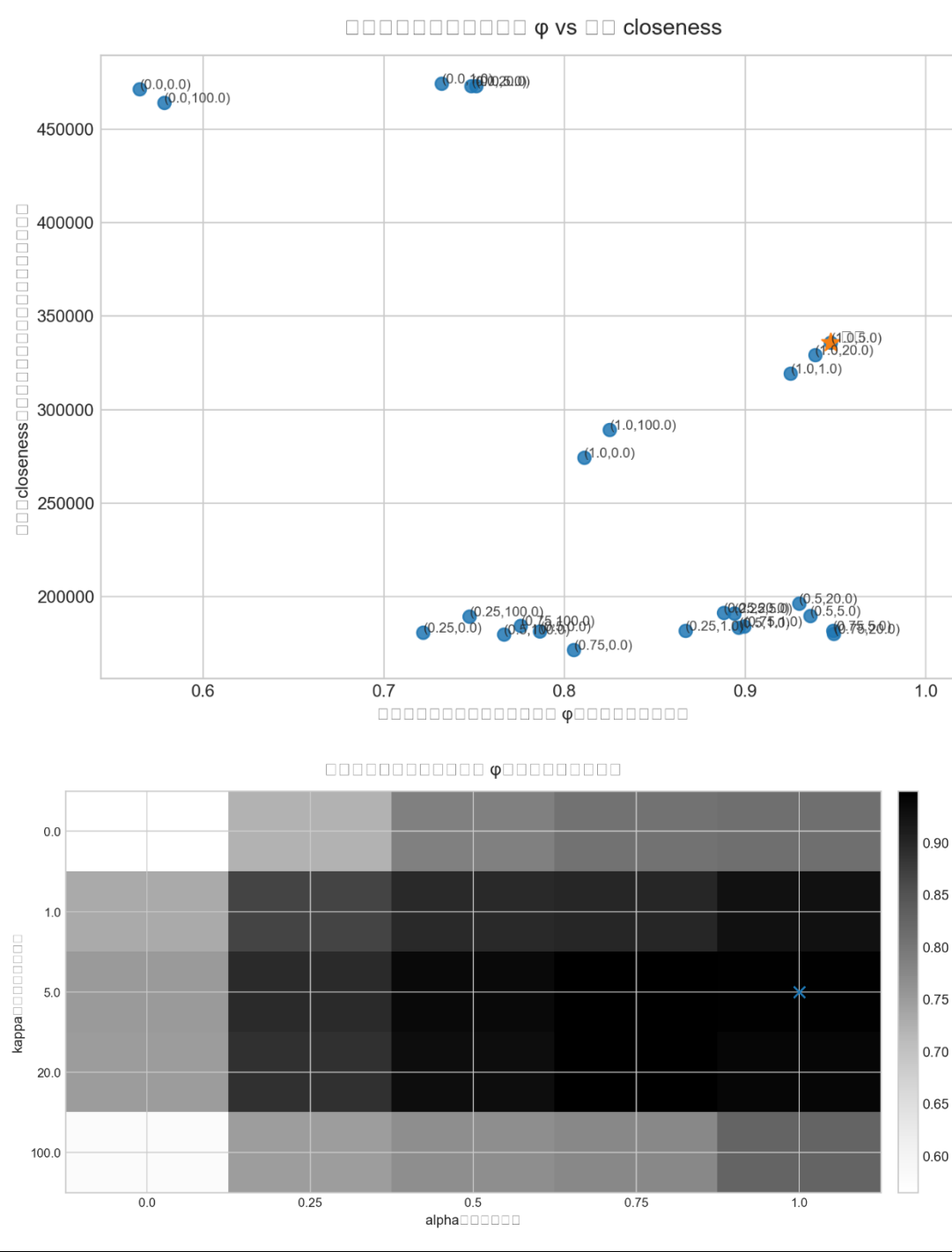
3.2 参数空间搜索与制度性能评估

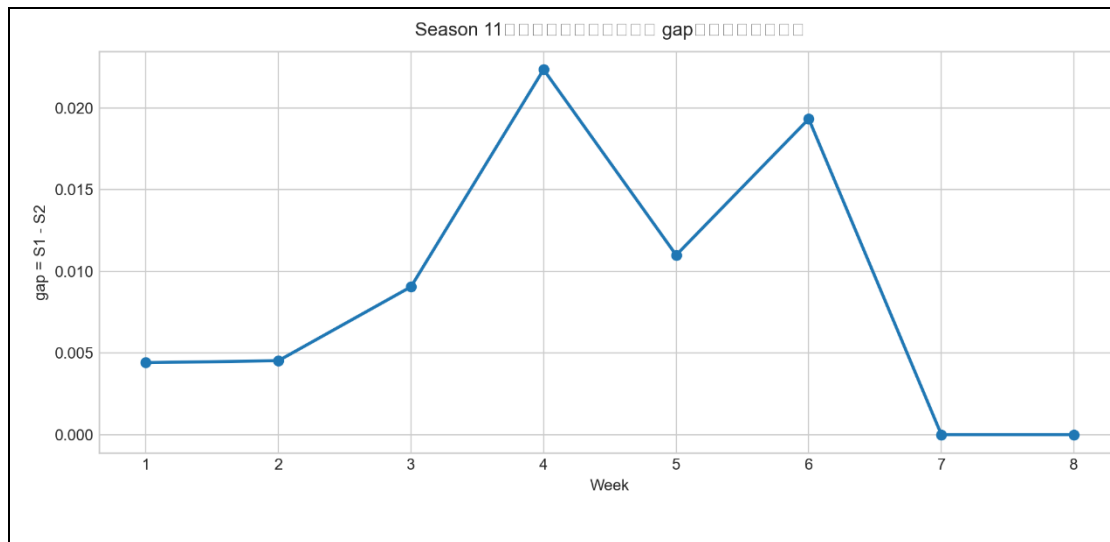
在合理区间内对 (α, κ) 进行网格搜索。对每一组参数组合，计算全体赛季的平均公平性指标 $\bar{\phi}(\alpha, \kappa)$ 以及吸引力指标 $\mathcal{U}(\alpha, \kappa)$ 。通过绘制 $\bar{\phi}$ 与 $\mathcal{U}$ 的权衡关系，可以直观展示制度在“更公平”与“更有悬念”之间的折中区域，而不是给出单一的主观最优解。

3.3 争议案例的定向验证与结论生成

最后，将第 2 问中提及的争议赛季作为重点样本，比较其在原制度与新制度下的结果差异。如果在一组温和参数下，新制度能显著降低争议人物夺冠或进入决赛的概率，而整体

赛季仍保持较高的不确定性，则可以据此给出明确建议：相较于单纯调整权重，新制度通过非线性处理与二阶段裁决，在结构上更适合同时满足技术公平性与节目吸引力的双重目标。





Python 代码:

```
# -*- coding: utf-8 -*-
"""
```

问题 4: 设计并评估“新评分制度”

核心制度（与前文思路一致）：

- 第一阶段（合成打分）： $S = a * q + (1-a) * f(p)$ ，其中 $f(p) = \sqrt{p}$ （抑制极端人气的线性放大）
- 第二阶段（危险区评委裁决）：先选 *Bottom2* 进入危险区，再由评委更可能淘汰技术更弱者

$$P(\text{淘汰 } a \mid \{a, b\}) \propto \exp(-\kappa * J_a)$$

评估目标（用历史赛季反事实回放）：

- 公平性（尊重技术）：淘汰者在当周评委名次中的分位数 ϕ ， ϕ 越接近 1 越“淘汰评委低分者”
- 吸引力/悬念（不确定性）：用“当周合成分前二名的差距”衡量；差距越小越有悬念

$$\text{closeness} = \text{mean}(1 / (\text{gap} + \text{eps}))$$

实现要求：

- 读取真实数据：2026_MCM_Problem_C_Data.csv（根目录，直接用文件名加载）
- 优先读取问题 1 投票估计：result1/q1_estimated_votes_long.csv（含 p_{hat} 、 q_{hat} 、 J ）
若不存在，则自动快速估计 p_{hat} （用百分比淘汰一致性约束近似，确保能跑通）
- 图像保存到 result4 文件夹（先创建），不展示
- 输出若干 CSV 方便写报告
- 注释尽量详细（中文）

```
"""
```

```

import os
import re
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 若缺少 Q1 投票估计, 会用到凸优化来快速估计  $p_{\hat{}}$ 
# pip install cvxpy
import cvxpy as cp

# =====
# 0. 输出目录与绘图风格
# =====
RESULT_DIR = "result4"
os.makedirs(RESULT_DIR, exist_ok=True)

plt.style.use("seaborn-v0_8-whitegrid")
np.random.seed(42)

# =====
# 1. 读取真实数据并解析结构
# =====
DATA_FILE = "2026_MCM_Problem_C_Data.csv"
df = pd.read_csv(DATA_FILE)
cols = df.columns.tolist()

def find_col(candidates, columns):
    """尽量稳健地识别列名（兼容大小写）"""
    for c in candidates:
        if c in columns:
            return c
    cols_lower = {x.lower(): x for x in columns}
    for c in candidates:
        if c.lower() in cols_lower:
            return cols_lower[c.lower()]
    raise ValueError(f"找不到列: 候选={candidates}")

season_col = find_col(["season", "Season", "SEASON"], cols)

try:
    contestant_col = find_col(

```

```

        ["celebrity", "Celebrity", "contestant", "Contestant",
        "star", "Star", "name", "Name"],
        cols
    )
except Exception:
    obj_cols = [c for c in cols if df[c].dtype == "object"]
    if not obj_cols:
        raise ValueError("数据里没有明显文本列可作为选手标识。请检查 CSV。")
    contestant_col = sorted(obj_cols, key=lambda c:
df[c].nunique(), reverse=True)[0]
    warnings.warn(f"未识别到标准选手列名，暂用 {contestant_col} 作为选手标识列")

# weekX_judgeY_score
week_judge_pattern = re.compile(r"week(\d+)_judge(\d+)_score",
re.IGNORECASE)
week_cols = {}
for c in cols:
    m = week_judge_pattern.match(c)
    if m:
        w = int(m.group(1))
        week_cols.setdefault(w, []).append(c)
if not week_cols:
    raise ValueError("未找到 weekX_judgeY_score 列，请检查 CSV 格式。")

all_weeks = sorted(week_cols.keys())

# 计算每周评委总分 J_week{w}
for w in all_weeks:
    df[f"J_week{w}"] = df[week_cols[w]].apply(pd.to_numeric,
errors="coerce").sum(axis=1, skipna=True)

# 每赛季有效周：该周至少有一人 J>0
season_weeks = {}
for s, g in df.groupby(season_col):
    valid_ws = []
    for w in all_weeks:
        if (g[f"J_week{w}"] > 0).any():
            valid_ws.append(w)
    season_weeks[int(s)] = valid_ws

```

```

# =====
# 2. 若 Q1 投票估计不存在: 快速估计  $p\_hat$  (保证本脚本可独立运行)
# =====
def infer_eliminations_one_season(g_season: pd.DataFrame,
valid_weeks):
    """
    使用题面“淘汰后后续周得分记为 0”的机制推断淘汰:
    本周  $J > 0$  且 下周  $J == 0 \Rightarrow$  本周淘汰
    返回 week  $\rightarrow$  [淘汰者名单] (可能为空/多个)
    """
    elim_map = {}
    names = g_season[contestant_col].astype(str).values

    for idx in range(len(valid_weeks) - 1):
        w = valid_weeks[idx]
        w_next = valid_weeks[idx + 1]
        alive_w = g_season[f"J_week{w}"].values > 0
        alive_next = g_season[f"J_week{w_next}"].values > 0
        eliminated = names[(alive_w) & (~alive_next)]
        elim_map[w] = list(eliminated)

    elim_map[valid_weeks[-1]] = []
    return elim_map

def solve_votes_one_season_percent_approx(
    g_season: pd.DataFrame,
    valid_weeks,
    elim_map,
    lam_entropy=0.60,
    lam_smooth=0.40,
    smooth_weight=5.0,
    eps_margin=1e-6,
):
    """
    快速估计投票占比  $p\_hat$  (用于后续制度回放):
    - 约束: 每周  $p \geq 0$  且  $\sum(p) = 1$ 
    - 淘汰一致性 (近似):  $T = q + p$ , 真实淘汰者  $T$  最小 (支持多淘汰)
    - 目标: 最大熵 (最少偏置) + 跨周平滑 (避免无理由剧烈跳变)
    输出: 长表 season, week, contestant, J, q_hat, p_hat, T_hat
    """
    season_id = int(g_season[season_col].iloc[0])
    contestants = g_season[contestant_col].astype(str).tolist()
    n_total = len(contestants)

```

```

# 每周存活选手、J
week_alive_names = {}
week_J = {}
for w in valid_weeks:
    alive_mask = g_season[f"J_week{w}"].values > 0
    alive_names = [contestants[i] for i in range(n_total) if
alive_mask[i]]
    week_alive_names[w] = alive_names
    week_J[w] = g_season.loc[alive_mask,
f"J_week{w}"].values.astype(float)

p_vars = {}
constraints = []
obj_terms = []

for w in valid_weeks:
    m = len(week_alive_names[w])
    p = cp.Variable(m, nonneg=True)
    p_vars[w] = p
    constraints.append(cp.sum(p) == 1)
    # 最大熵: 最小化  $\sum p \log p = -\sum \text{entr}(p)$ 
    obj_terms.append(lam_entropy * cp.sum(-cp.entr(p)))

# 淘汰一致性约束 (近似百分比合并)
for w in valid_weeks:
    elims = elim_map.get(w, [])
    if not elims:
        continue
    alive_names = week_alive_names[w]
    m = len(alive_names)
    if m <= 1:
        continue

    J = week_J[w]
    J_sum = float(np.sum(J)) if np.sum(J) > 0 else 1.0
    q = J / J_sum

    T = q + p_vars[w]

    elim_idx = [alive_names.index(nm) for nm in elims if nm in
alive_names]
    if not elim_idx:
        continue

```

```

surv_idx = [i for i in range(m) if i not in elim_idx]
if not surv_idx:
    continue

for ei in elim_idx:
    constraints.append(T[ei] <= T[surv_idx] - eps_margin)

# 跨周平滑: 相邻周都在赛的一选手 p 尽量接近
for i in range(1, len(valid_weeks)):
    w_prev = valid_weeks[i - 1]
    w_curr = valid_weeks[i]
    prev_names = week_alive_names[w_prev]
    curr_names = week_alive_names[w_curr]
    common = sorted(set(prev_names).intersection(curr_names))
    if not common:
        continue

    diffs = []
    for nm in common:
        ip = prev_names.index(nm)
        ic = curr_names.index(nm)
        diffs.append(p_vars[w_curr][ic] - p_vars[w_prev][ip])

    diffs = cp.hstack(diffs)
    obj_terms.append(lam_smooth * smooth_weight *
cp.sum_squares(diffs))

prob = cp.Problem(cp.Minimize(cp.sum(obj_terms)), constraints)

solved = False
last_err = None
for solver in ["ECOS", "SCS", "OSQP"]:
    try:
        prob.solve(solver=solver, verbose=False)
        if prob.status in ["optimal", "optimal_inaccurate"]:
            solved = True
            break
    except Exception as e:
        last_err = e

if not solved:
    raise RuntimeError(f"赛季{season_id} 投票估计失败:
{last_err}")

```

```

rows = []
for w in valid_weeks:
    alive_names = week_alive_names[w]
    J = week_J[w]
    J_sum = float(np.sum(J)) if np.sum(J) > 0 else 1.0
    q = J / J_sum
    p_hat = np.maximum(p_vars[w].value, 0)
    p_hat = p_hat / (p_hat.sum() if p_hat.sum() > 0 else 1.0)
    T_hat = q + p_hat

    for k, nm in enumerate(alive_names):
        rows.append({
            "season": season_id,
            "week": int(w),
            "contestant": str(nm),
            "J": float(J[k]),
            "q_hat": float(q[k]),
            "p_hat": float(p_hat[k]),
            "T_hat": float(T_hat[k]),
        })

return pd.DataFrame(rows)

# =====
# 3. 读取 (或生成) Q1 投票估计 votes_all
# =====
Q1_FILE = os.path.join("result1", "Q1_estimated_votes_long.csv")

if os.path.exists(Q1_FILE):
    votes_all = pd.read_csv(Q1_FILE)
    # 简单兼容大小写
    col_map = {}
    for c in votes_all.columns:
        if c.lower() == "season": col_map[c] = "season"
        if c.lower() == "week": col_map[c] = "week"
        if c.lower() == "contestant": col_map[c] = "contestant"
        if c.lower() == "j": col_map[c] = "J"
        if c.lower() == "q_hat": col_map[c] = "q_hat"
        if c.lower() == "p_hat": col_map[c] = "p_hat"
        if c.lower() == "t_hat": col_map[c] = "T_hat"
    votes_all = votes_all.rename(columns=col_map)
    print(f"已读取问题 1 投票估计: {Q1_FILE}")
else:

```

```

    print("未找到 result1/Q1_estimated_votes_long.csv, 将自动估计
    p_hat (可运行但会更慢)。")
    all_votes = []
    for s in sorted(season_weeks.keys()):
        g = df[df[season_col] == s].copy()
        valid_ws = season_weeks[int(s)]
        if len(valid_ws) < 2:
            continue
        elim_map = infer_eliminations_one_season(g, valid_ws)
        votes_long = solve_votes_one_season_percent_approx(g,
        valid_ws, elim_map)
        all_votes.append(votes_long)
        votes_all = pd.concat(all_votes, ignore_index=True)
        # 也存一份到 result4, 便于复用
        votes_all.to_csv(os.path.join(RESULT_DIR,
        "Q1_estimated_votes_long_copy.csv"), index=False, encoding="utf-8-
        sig")
        print(f"已在 {RESULT_DIR} 中输出投票估计副本。")

    votes_all["season"] = votes_all["season"].astype(int)
    votes_all["week"] = votes_all["week"].astype(int)
    votes_all["contestant"] = votes_all["contestant"].astype(str)
    for c in ["J", "q_hat", "p_hat"]:
        votes_all[c] = pd.to_numeric(votes_all[c], errors="coerce")

# =====
# 4. 新制度回放函数 (α, κ 可调)
# =====
def new_rule_score(q, p, alpha):
    """
    第一阶段合成分:  $S = \alpha * q + (1 - \alpha) * \sqrt{p}$ 
    说明:
    - q 是评委百分比 (当周总分归一化)
    - p 是粉丝投票占比 (已是当周归一化)
    -  $\sqrt{p}$  是“抑制极端人气线性放大”的关键
    """
    return alpha * q + (1.0 - alpha) * np.sqrt(np.maximum(p, 0.0))

def judge_save_pick(bottom2_df, kappa, rng):
    """
    在倒二集合中由评委裁决淘汰:
     $P(\text{淘汰 } a) \propto \exp(-\kappa * J_a)$ 

```

```

- kappa 越大, 越接近 “必淘汰评委分更低者”
"""

names = bottom2_df["contestant"].tolist()
Js = bottom2_df["J"].astype(float).values

if len(names) == 1:
    return names[0]

# softmax(-kappa*J)
w = np.exp(-kappa * Js)
s = w.sum()
if s <= 0:
    return rng.choice(names)

probs = w / s
return rng.choice(names, p=probs)

def simulate_season_new_rule(votes_season, alpha=0.5, kappa=20.0,
rng=None):
    """
    对单赛季进行“新制度”回放:
    - 每周基于存活集合计算 S
    - 选 Bottom2 进入危险区
    - 由评委概率机制选出最终淘汰者
    输出:
    - elim_pred: week -> 淘汰者 (单淘汰版本; 若你要多淘汰, 可再扩展)
    - logs: 每周存活选手的 S、J、q、p, 用于计算指标与画图
    - final_rank: 最后一周按 S 从高到低排序 (冠军第一)
    """
    if rng is None:
        rng = np.random.default_rng(123)

    weeks = sorted(votes_season["week"].unique().tolist())
    if len(weeks) < 2:
        return {}, pd.DataFrame(), []

    # 初始存活 = 第一周出现的选手
    alive = set(votes_season[votes_season["week"] ==
weeks[0]]["contestant"].tolist())

    elim_pred = {}
    logs = []

```

```

# 逐周淘汰直到最后一周（最后一周只排名，不淘汰）
for w in weeks[:-1]:
    wdf = votes_season[votes_season["week"] == w].copy()
    wdf = wdf[wdf["contestant"].isin(alive)].copy()
    if wdf.empty or len(alive) <= 1:
        break

    # 计算新制度第一阶段合成分
    wdf["S"] = new_rule_score(
        wdf["q_hat"].astype(float).values,
        wdf["p_hat"].astype(float).values,
        alpha=alpha
    )

    # 危险区：S 最低的两人
    wdf_sorted = wdf.sort_values("S", ascending=True)
    bottom2 = wdf_sorted.head(2).copy()

    # 评委裁决淘汰
    eliminated = judge_save_pick(bottom2, kappa=kappa, rng=rng)

    # 记录日志
    wdf_log = wdf.copy()
    wdf_log["alive_n"] = len(alive)
    wdf_log["week"] = w
    logs.append(wdf_log)

    # 更新存活集合
    elim_pred[w] = eliminated
    if eliminated in alive:
        alive.remove(eliminated)

# 决赛周排名（按 S 从高到低）
last_w = weeks[-1]
last_df = votes_season[votes_season["week"] == last_w].copy()
last_df = last_df[last_df["contestant"].isin(alive)].copy()
if last_df.empty:
    final_rank = []
else:
    last_df["S"] = new_rule_score(
        last_df["q_hat"].astype(float).values,
        last_df["p_hat"].astype(float).values,
        alpha=alpha
    )

```

```

    )
    last_df = last_df.sort_values("S", ascending=False)
    final_rank = last_df["contestant"].tolist()

    logs_df = pd.concat(logs, ignore_index=True) if logs else
pd.DataFrame()
    return elim_pred, logs_df, final_rank

# =====
# 5. 指标计算: 公平性  $\phi$  + 悬念 closeness
# =====
def compute_phi_for_elims(logs_df, elim_pred):
    """
     $\phi = (rankJ - 1)/(n-1)$ , rankJ=1 表示评委最好
     $\phi$  越大 => 淘汰者越接近评委最差 => 更尊重技术评价
    """
    rows = []
    if logs_df.empty:
        return pd.DataFrame(rows)

    for w, g in logs_df.groupby("week"):
        g = g.copy()
        n = len(g)
        if n <= 1:
            continue

        # 评委名次: J 越大越好, rank=1 最好
        g["rankJ"] = (-
g["J"].astype(float)).rank(method="min").astype(int)

        e = elim_pred.get(int(w), None)
        if e is None or e not in g["contestant"].values:
            continue

        rj = int(g.loc[g["contestant"] == e, "rankJ"].iloc[0])
        phi = (rj - 1) / (n - 1)
        rows.append({"week": int(w), "eliminated": e, "rankJ": rj,
"alive_n": n, "phi": phi})
    return pd.DataFrame(rows)

def compute_closeness(logs_df, eps=1e-6):
    """

```

悬念度 (*closeness*) :

- 当周按 *S* 从高到低排序, 取前二名差距 $gap = S1 - S2$

- $closeness = mean(1/(gap+eps))$

gap 越小 => *closeness* 越大 => 越有悬念

"""

```
if logs_df.empty or "S" not in logs_df.columns:
    return np.nan
```

```
closeness_list = []
for w, g in logs_df.groupby("week"):
    g = g.dropna(subset=["S"]).copy()
    if len(g) < 2:
        continue
    g = g.sort_values("S", ascending=False)
    gap = float(g["S"].iloc[0] - g["S"].iloc[1])
    closeness_list.append(1.0 / (gap + eps))
if not closeness_list:
    return np.nan
return float(np.mean(closeness_list))
```

=====

6. 网格搜索: 在历史赛季上评估 (α , κ) 的平均表现

=====

```
alphas = [0.0, 0.25, 0.5, 0.75, 1.0]
kappas = [0.0, 1.0, 5.0, 20.0, 100.0]
```

```
rng_master = np.random.default_rng(2026)
```

```
grid_rows = []
season_detail_rows = []
```

```
seasons = sorted(votes_all["season"].unique().tolist())
```

```
for alpha in alphas:
    for kappa in kappas:
        phi_vals = []
        close_vals = []
        n_elim_weeks_total = 0

        for s in seasons:
            vs = votes_all[votes_all["season"] == s].copy()
            if vs["week"].nunique() < 2:
                continue
```

```

        elim_pred, logs_df, final_rank =
simulate_season_new_rule(
    vs, alpha=alpha, kappa=kappa, rng=rng_master
)

# 计算  $\phi$ 
phi_df = compute_phi_for_elims(logs_df, elim_pred)
if not phi_df.empty:
    phi_vals.append(phi_df["phi"].mean())
    n_elim_weeks_total += len(phi_df)

# 计算悬念 closeness
close = compute_closeness(logs_df)
if not np.isnan(close):
    close_vals.append(close)

# 存一份“赛季级明细”（用于报告举例）
season_detail_rows.append({
    "season": int(s),
    "alpha": alpha,
    "kappa": kappa,
    "phi_mean_season": float(phi_df["phi"].mean()) if
not phi_df.empty else np.nan,
    "closeness_season": float(close) if not
np.isnan(close) else np.nan,
    "champion": final_rank[0] if final_rank else ""
})

grid_rows.append({
    "alpha": alpha,
    "kappa": kappa,
    "phi_mean": float(np.nanmean(phi_vals)) if phi_vals
else np.nan,
    "closeness_mean": float(np.nanmean(close_vals)) if
close_vals else np.nan,
    "n_elim_weeks_total": int(n_elim_weeks_total)
})

grid_df = pd.DataFrame(grid_rows)
detail_df = pd.DataFrame(season_detail_rows)

grid_csv = os.path.join(RESULT_DIR,
"Q4_grid_metrics_alpha_kappa.csv")

```

```

grid_df.to_csv(grid_csv, index=False, encoding="utf-8-sig")

detail_csv = os.path.join(RESULT_DIR,
"Q4_season_detail_metrics.csv")
detail_df.to_csv(detail_csv, index=False, encoding="utf-8-sig")

# =====
# 7. 选择一个“推荐参数”（简单做法：公平性与悬念都尽量高）
# =====
# 这里用一个很实用的综合评分： $z(\phi) + z(closeness)$ 
tmp = grid_df.dropna(subset=["phi_mean", "closeness_mean"]).copy()
if not tmp.empty:
    tmp["phi_z"] = (tmp["phi_mean"] - tmp["phi_mean"].mean()) /
(tmp["phi_mean"].std() + 1e-12)
    tmp["close_z"] = (tmp["closeness_mean"] -
tmp["closeness_mean"].mean()) / (tmp["closeness_mean"].std() + 1e-
12)
    tmp["score"] = tmp["phi_z"] + tmp["close_z"]
    best = tmp.sort_values("score", ascending=False).iloc[0]
    best_alpha = float(best["alpha"])
    best_kappa = float(best["kappa"])
else:
    best_alpha, best_kappa = 0.5, 20.0

best_txt = os.path.join(RESULT_DIR, "Q4_recommended_params.txt")
with open(best_txt, "w", encoding="utf-8") as f:
    f.write(f"推荐参数（基于  $z(\phi)+z(closeness)$  最大）：
alpha={best_alpha}, kappa={best_kappa}\n")

# =====
# 8. 可视化（全部保存，不展示）
# =====
def save_heatmap(df_grid, value_col, title, outfile):
    """画  $\alpha$ - $\kappa$  的热力图（不指定花哨风格也能清晰）"""
    pivot = df_grid.pivot(index="kappa", columns="alpha",
values=value_col)
    pivot = pivot.reindex(index=sorted(pivot.index),
columns=sorted(pivot.columns))

    fig, ax = plt.subplots(figsize=(10, 5), dpi=200)
    im = ax.imshow(pivot.values, aspect="auto",
interpolation="nearest")

```

```

ax.set_title(title, fontsize=13, pad=12)
ax.set_xlabel("alpha (评委权重)")
ax.set_ylabel("kappa (评委裁决强度)")

ax.set_xticks(np.arange(len(pivot.columns)))
ax.set_xticklabels([str(a) for a in pivot.columns], fontsize=9)

ax.set_yticks(np.arange(len(pivot.index)))
ax.set_yticklabels([str(k) for k in pivot.index], fontsize=9)

cbar = fig.colorbar(im, ax=ax, fraction=0.03, pad=0.02)
cbar.set_label(value_col)

# 标注最佳点 (如果该值存在)
if value_col in ["phi_mean", "closeness_mean"]:
    try:
        a_idx = list(pivot.columns).index(best_alpha)
        k_idx = list(pivot.index).index(best_kappa)
        ax.scatter([a_idx], [k_idx], s=70, marker="x")
    except Exception:
        pass

fig.tight_layout()
fig.savefig(os.path.join(RESULT_DIR, outfile),
bbox_inches="tight")
plt.close(fig)

def save_pareto_scatter(df_grid, outfile):
    """画公平性-悬念散点 (每个点是一组(alpha, kappa)) """
    d = df_grid.dropna(subset=["phi_mean",
"closeness_mean"]).copy()
    if d.empty:
        return

    fig, ax = plt.subplots(figsize=(8, 6), dpi=200)
    ax.scatter(d["phi_mean"], d["closeness_mean"], s=55,
alpha=0.85)

    # 标注每个点对应的(alpha, kappa) (点不多, 标注可读性还可以)
    for _, r in d.iterrows():
        ax.text(r["phi_mean"], r["closeness_mean"],
f"({r['alpha']}, {r['kappa']})", fontsize=8, alpha=0.85)

```

```

ax.set_title("新制度参数权衡：公平性  $\phi$  vs 悬念 closeness",
fontsize=13, pad=12)
ax.set_xlabel("公平性：淘汰者评委分位数均值  $\phi$ （越高越尊重评委）")
ax.set_ylabel("悬念：closeness（越高表示前二差距越小、越有悬念）")

# 标注推荐点
ax.axvline(best_alpha, alpha=0.0) # 占位避免误会
best_row = d[(d["alpha"] == best_alpha) & (d["kappa"] ==
best_kappa)]
if not best_row.empty:
    bx = float(best_row["phi_mean"].iloc[0])
    by = float(best_row["closeness_mean"].iloc[0])
    ax.scatter([bx], [by], s=120, marker="*", alpha=0.95)
    ax.text(bx, by, " 推荐", fontsize=10, weight="bold")

fig.tight_layout()
fig.savefig(os.path.join(RESULT_DIR, outfile),
bbox_inches="tight")
plt.close(fig)

def save_focus_season_comparison(season_id, alpha, kappa, outfile):
    """
    对某个赛季：画每周的
    - 合成分  $S$  的分布（用淘汰者  $S$  位置）
    - 淘汰者评委分位数  $\phi$  随周变化
    这样更像“节目方可读”的证据图
    """
    vs = votes_all[votes_all["season"] == season_id].copy()
    if vs.empty or vs["week"].nunique() < 2:
        return

    rng_local = np.random.default_rng(7)
    elim_pred, logs_df, final_rank = simulate_season_new_rule(vs,
alpha=alpha, kappa=kappa, rng=rng_local)
    if logs_df.empty:
        return

    # 计算每周淘汰者  $\phi$ 
    phi_df = compute_phi_for_elims(logs_df, elim_pred)
    if phi_df.empty:

```

```

return

# 图 1: phi 随周变化
fig, ax = plt.subplots(figsize=(10, 4.5), dpi=200)
ax.plot(phi_df["week"], phi_df["phi"], marker="o", linewidth=2)
ax.set_title(f"Season {season_id}: 新制度下淘汰者评委分位数  $\phi$  (alpha={alpha}, kappa={kappa})", fontsize=12, pad=10)
ax.set_xlabel("Week")
ax.set_ylabel(" $\phi$  (越接近 1 越淘汰评委低分者)")
ax.set_ylim(-0.05, 1.05)
fig.tight_layout()
fig.savefig(os.path.join(RESULT_DIR, outfile.replace(".png", "_phi.png")), bbox_inches="tight")
plt.close(fig)

# 图 2: 每周“前二差距 gap”随周变化 (体现悬念)
gap_rows = []
for w, g in logs_df.groupby("week"):
    g = g.dropna(subset=["S"]).copy()
    if len(g) < 2:
        continue
    g = g.sort_values("S", ascending=False)
    gap = float(g["S"].iloc[0] - g["S"].iloc[1])
    gap_rows.append({"week": int(w), "gap_top1_top2": gap})
gap_df = pd.DataFrame(gap_rows)
if not gap_df.empty:
    fig, ax = plt.subplots(figsize=(10, 4.5), dpi=200)
    ax.plot(gap_df["week"], gap_df["gap_top1_top2"],
marker="o", linewidth=2)
    ax.set_title(f"Season {season_id}: 新制度下每周前二差距 gap (越小越有悬念)", fontsize=12, pad=10)
    ax.set_xlabel("Week")
    ax.set_ylabel("gap = S1 - S2")
    fig.tight_layout()
    fig.savefig(os.path.join(RESULT_DIR,
outfile.replace(".png", "_gap.png")), bbox_inches="tight")
    plt.close(fig)

# 8.1 热力图: 公平性 & 悬念
save_heatmap(grid_df, "phi_mean", "新制度公平性热力图: 平均  $\phi$  (越高越尊重评委)", "Q4_heatmap_phi.png")
save_heatmap(grid_df, "closeness_mean", "新制度悬念热力图: closeness (越高越有悬念)", "Q4_heatmap_closeness.png")

```

```

# 8.2 Pareto 散点 (权衡)
save_pareto_scatter(grid_df, "Q4_pareto_phi_vs_closeness.png")

# 8.3 画几个重点赛季 (如果存在)
for focus in [2, 11, 27, 28, 34]:
    if focus in seasons:
        save_focus_season_comparison(
            season_id=focus,
            alpha=best_alpha,
            kappa=best_kappa,
            outfile=f"Q4_focus_season_{focus}.png"
        )

# =====
# 9. 输出一份“推荐参数下”的赛季级汇总 (写结论很方便)
# =====
rec = detail_df[(detail_df["alpha"] == best_alpha) &
                (detail_df["kappa"] == best_kappa)].copy()
rec_out = os.path.join(RESULT_DIR,
                       "Q4_recommended_params_season_summary.csv")
rec.to_csv(rec_out, index=False, encoding="utf-8-sig")

print("运行完成 ☑")

print(f"- 网格评估指标: {grid_csv}")
print(f"- 赛季明细指标: {detail_csv}")
print(f"- 推荐参数: {best_txt}")
print(f"- 推荐参数赛季汇总: {rec_out}")
print(f"- 图片输出目录: {RESULT_DIR}")
print("主要图片: ")
print("  - Q4_heatmap_phi.png")
print("  - Q4_heatmap_closeness.png")
print("  - Q4_pareto_phi_vs_closeness.png")
print("  - Q4_focus_season_*_phi.png / Q4_focus_season_*_gap.png")
    (若对应赛季存在)

```